

# The Ultimate Self-Contained Python Study Guide & Textbook

---

Welcome to the ultimate Python programming masterclass reference guide. This document is fully self-contained and meticulously detailed. If you have a basic understanding of programming and want to learn everything Python has to offer—from the absolute fundamentals to advanced object-oriented architectures, GUI design, file operations, multi-threading, API integration, and dozens of fully implemented games and tools—**this single document is all you need.**

Each topic is explained thoroughly, accompanied by clean code examples, logical breakdowns, and complete, production-ready, hands-on projects placed chronologically exactly where they appear in the course.

---

## Table of Contents

---

0. Module 0: Bridging From C/C++ to Python (Read This First)
1. Module 1: Setup & Python Basics
2. Module 2: User Input & Initial Projects
3. Module 3: Math and Circle Geometry Projects
4. Module 4: Control Flow, Decision-Making, & Calculator Projects
5. Module 5: Logical Operators & Ternary Shortcuts
6. Module 6: String Methods & Username Input Validation
7. Module 7: String Indexing, Slicing, & Credit Card Masking
8. Module 8: Format Specifiers inside F-Strings
9. Module 9: While Loops & Compound Interest Calculator
10. Module 10: For Loops & Digital Countdown Timer
11. Module 11: Nested Loops & Symbol Rectangle Printer
12. Module 12: Core Collections & Interactive Shopping Cart
13. Module 13: 2D Collections & Telephone Keypad Generator
14. Module 14: Interactive Science Quiz Game Project
15. Module 15: Dictionaries & Movie Theater Concession Stand
16. Module 16: Random Module & Number Guessing Game
17. Module 17: Rock, Paper, Scissors Game Project
18. Module 18: Dice Roller with ASCII Art Project
19. Module 19: Functions, Parameter Rules, & Default/Keyword Args
20. Module 20: Arbitrary Arguments (*args*, *\*kwargs*) & Shipping Label Generator
21. Module 21: Iterables & Membership Operators
22. Module 22: List Comprehensions & Match-Case Statements
23. Module 23: Modules, Scope Resolution (LEGB), & the Entry Point
24. Module 24: Robust CLI ATM Banking Project
25. Module 25: Classic Slot Machine Simulator Project
26. Module 26: Substitution Cipher Encryption System Project
27. Module 27: Hangman Game Project with Gallows ASCII Art
28. Module 28: Object-Oriented Programming (OOP) Deep Dive
29. Module 29: Advanced Inheritance & Super() Shape Exercises
30. Module 30: Polymorphism, Duck Typing, & Static/Class Methods
31. Module 31: Magic/Dunder Methods & Property Decorators
32. Module 32: Decorators & Exception Handling
33. Module 33: File Detection & File I/O (Plaintext, JSON, CSV)
34. Module 34: Datetime & Alarm Clock Project
35. Module 35: Multi-threading & API Pokémon Integration
36. Module 36: PyQt5 GUI Development Fundamentals
37. Module 37: PyQt5 Widgets (Buttons, Checkboxes, Radios, Textboxes)
38. Module 38: PyQt5 Digital Clock Widget Project
39. Module 39: PyQt5 Precision Stopwatch App Project
40. Module 40: PyQt5 Real-Time Weather Application Project

## Module 0: Bridging From C/C++ to Python (Read This First)

You already know how to program. This section exists purely to translate what you know into Python's mental model, so you stop tripping over things that "should" work like C/C++ but don't.

### 1. No Compilation Step

C/C++ is compiled to machine code ahead of time; the compiler catches type errors before the program ever runs. Python is **interpreted line-by-line at runtime** by default (technically compiled to bytecode on the fly, but there's no separate build step you manage). This means:

\* Type errors, typos in variable names, etc. are only caught when that line actually executes — not when you "compile."

\* A syntax error deep inside a function you never call will not be caught until that function runs (or, for basic syntax errors, when the file is parsed).

### 2. Dynamic Typing, Not Static Typing

In C++, `int x = 5;` permanently binds the name `x` to the type `int`. In Python, `x = 5` just points the name `x` at an integer object. The **same name can be rebound to a completely different type** later — there is no compiler to stop you:

```
x = 5          # x refers to an int
x = "hello"    # now x refers to a str — perfectly legal
```

Python variables are **labels/references**, not typed memory boxes. This single idea explains a lot of downstream behavior (see point 3).

### 3. Variables Are References, Not Boxes (Your "Pointers" Instinct Still Applies)

This is the single biggest mental model shift coming from C/C++.

In C++, `int a = 5; int b = a;` copies the value — `a` and `b` are independent.

In Python, assignment **never copies the object**; it just makes another name point to the same object.

```
list_a = [1, 2, 3]
list_b = list_a      # list_b points to the SAME list object as list_a
list_b.append(4)
print(list_a)        # Outputs: [1, 2, 3, 4] <- list_a changed too!
```

This is exactly the behavior you'd get in C++ if `a` and `b` were both pointers to the same heap-allocated array. Python just hides the `*` and `&` — but the underlying reference semantics are the same.

- **Immutable types** (`int`, `float`, `str`, `bool`, `tuple`) behave like values because you can never modify them in place — any "change" creates a brand-new object and rebinds the name to it. So aliasing is invisible for these types.
- **Mutable types** (`list`, `dict`, `set`, and any custom object) can be changed in place, so aliasing is very visible — as shown above.
- To actually copy a mutable object, use `.copy()`, `list(original)`, or `import copy; copy.deepcopy(original)` for nested structures. See the note in Module 12 for details.

### 4. No Pointers, No Manual Memory Management

There is no `malloc` / `free`, no `new` / `delete`, no dangling pointers, and no manual memory management at all. Python uses automatic **reference counting + garbage collection**: once nothing refers to an object anymore, its memory is reclaimed for you. You cannot leak memory the way you can in C (though you can still hold references longer than intended and bloat memory usage).

### 5. Indentation Replaces `{ }` and `;`

There are no curly braces and no semicolons. **Indentation is not a style**

**preference — it is the syntax that defines a block.** Mixing tabs and spaces, or inconsistent indentation, is a `SyntaxError` / `IndentationError`. Most editors (PyCharm included) auto-convert Tab to 4 spaces — keep it that way.

```
# C++: if (x > 0) { std::cout << x; }
# Python:
if x > 0:
    print(x)    # this indentation IS the block — not decoration
```

## 6. No Function/Method Overloading

C++ lets you define `add(int, int)` and `add(double, double)` side by side. Python does not support this — defining a function twice with the same name simply **replaces** the first definition. Instead, Python leans on:

- \* **Default arguments** (Module 19)
- \* `*args` / `**kwargs` (Module 20)
- \* **match - case style dispatch** or manual `isinstance()` checks
- \* **Duck typing** (Module 30) instead of type-based overload resolution

## 7. No Real Multi-Dimensional Arrays

C/C++ has true 2D arrays with contiguous memory ( `int grid[3][4]` ). Python's "2D lists" (Module 13) are really **lists of lists** — each row is its own independent list object. This has a nasty trap:

```
# WRONG — all 3 "rows" are the SAME list object (because * doesn't copy)
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)    # [[1, 0, 0], [1, 0, 0], [1, 0, 0]] <- every row changed!

# RIGHT — build independent row objects
grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 1
print(grid)    # [[1, 0, 0], [0, 0, 0], [0, 0, 0]] <- only row 0 changed
```

This is a direct consequence of point 3 (reference semantics) — worth re-reading if it doesn't click immediately.

## 8. `switch` Didn't Exist Until Recently

Python had no `switch` equivalent for decades — everyone used `if / elif` chains. Python 3.10 (2021) introduced `match - case` (Module 22), which is structurally closer to pattern matching (like Rust/Haskell) than a classic C `switch` — it can match on types, structure, and unpack values, not just constants.

## 9. Everything Is an Object (Even Functions and Classes Themselves)

In Python, functions, classes, and modules are themselves objects that can be assigned to variables, passed as arguments, and stored in collections. This isn't possible for functions in C (only via function pointers) and is worth previewing now — it's what makes decorators (Module 32) and passing functions as arguments feel natural rather than exotic.

## 10. Quick Terminology Cross-Reference

| C/C++ Concept                                                                                              | Python Equivalent                                                            | Notes                                                                                            |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>int</code> , <code>double</code> , <code>bool</code> , <code>char*</code> / <code>std::string</code> | <code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code> | No separate <code>char</code> type — a 1-character string is just a <code>str</code> of length 1 |
| <code>struct</code> / plain-data <code>class</code>                                                        | <code>class</code> (Module 28)                                               | Python classes are always more like C++ classes with public members by default                   |
| Pointers / references ( <code>*</code> , <code>&amp;</code> )                                              | Implicit — all variables are references                                      | See point 3                                                                                      |
| <code>#include</code>                                                                                      | <code>import</code> (Module 23)                                              | No header files; one <code>.py</code> file = one module                                          |
| Compiler errors                                                                                            | Runtime <code>Exception</code> s (Module 32)                                 | Caught with <code>try</code> / <code>except</code> , not at build time                           |
| <code>NULL</code> / <code>nullptr</code>                                                                   | <code>None</code>                                                            |                                                                                                  |
| <code>&amp;&amp;</code> , <code>\ \ </code> , <code>!</code>                                               | <code>and</code> , <code>or</code> , <code>not</code> (Module 5)             | Words, not symbols                                                                               |
| <code>==</code> (value equality only)                                                                      | <code>==</code> (value equality) vs <code>is</code> (identity)               | Python distinguishes these explicitly — see Module 1                                             |
| Function overloading                                                                                       | Default args / <code>*args</code> / duck typing                              | No true overloading                                                                              |
| <code>switch</code>                                                                                        | <code>match - case</code> (3.10+)                                            | Much more powerful — pattern matching                                                            |
| Arrays (fixed size, homogeneous)                                                                           | <code>list</code> (dynamic, heterogeneous)                                   | Also <code>tuple</code> for fixed/immutable sequences                                            |
| Manual memory mgmt                                                                                         | Automatic garbage collection                                                 | No <code>free</code> / <code>delete</code>                                                       |

## Module 1: Setup & Python Basics

### Python Installation & Environment Setup

To begin writing and executing Python programs, you need two fundamental software pieces:

- Python Interpreter:** Translates your plain-text Python code into machine code that your computer's processor can execute [1].  
\* *Installation on Windows:* Download the executable from [python.org](#). When running the setup file, **you must check the box "Add Python.exe to PATH"** before clicking "Install Now" [2]. This configures system environment variables to allow executing Python from any terminal command line [2].
- Integrated Development Environment (IDE):** The interface where you write, structure, and debug your program files.  
\* *PyCharm Community Edition:* A highly visual, beginner-friendly IDE [2]. You can download the free, open-source Community edition from JetBrains [2]. During installation, you can optionally create a desktop shortcut [3].  
\* *Creating a Project:* Launch PyCharm, select "New Project", configure your target directory, ensure the newly installed interpreter is selected, and create the project [3].  
\* *Creating a File:* Right-click your project directory -> **New -> Python File**. Name your main driver script `main.py` [3].

### Your First Program

To display messages to the output terminal (the console), Python uses the built-in `print()` function [4]. Inside the parentheses, you pass the data to be output [4]. Textual data (strings) must be wrapped inside double ( `"..."` ) or single ( `'...'` ) quotes [4].

```
# Printing simple lines of text to the console
print("I like pizza")
print("It's really good")
```

When PyCharm runs this code, it opens a console window and outputs the strings line-by-line [4].

### Comments in Python

Comments are non-executable annotations used to document your code for yourself or other developers [5]. The Python interpreter completely ignores them during runtime [4, 5].  
\* **Single-line comments:** Preceded by a `#` (hashtag/pound sign) [5].

```
# This is a comment. The interpreter skips this line.
print("Hello World!") # You can also write comments inline
```

### Variables & The 4 Fundamental Data Types

A **variable** is a named container that references a specific value stored in memory [5]. Once assigned, a variable behaves as if it were the value it represents [5]. Variables are assigned using the assignment operator ( `=` ) [5].

Python features four fundamental, beginner-friendly data types:

| Data Type | Description                                                                     | Python Syntax Code | Example                                                                   |
|-----------|---------------------------------------------------------------------------------|--------------------|---------------------------------------------------------------------------|
| String    | A series of text characters wrapped in quotes                                   | <code>str</code>   | <code>"Bro"</code> , <code>"pizza"</code> , <code>"user@email.com"</code> |
| Integer   | A whole number without a decimal portion (no quotes)                            | <code>int</code>   | <code>25</code> , <code>30</code> , <code>3</code>                        |
| Float     | A floating-point number containing a decimal portion                            | <code>float</code> | <code>3.14</code> , <code>10.99</code> , <code>3.2</code>                 |
| Boolean   | A binary logical state of <code>True</code> or <code>False</code> (capitalized) | <code>bool</code>  | <code>True</code> , <code>False</code>                                    |

## Data Type Demonstrations & F-Strings

To cleanly concatenate or inject variable values directly inside output strings without clunky formatting operators, Python uses **F-Strings (Formatted Strings)** [6]. Prefix your string quotes with the letter `f` (or `F`) and place your variables inside curly braces `{}` [6].

```
# 1. Strings (str)
first_name = "Bro"
food = "pizza"
email = "bro123@fake.com"

print(f"Hello {first_name}")           # Outputs: Hello Bro
print(f"You like {food}")              # Outputs: You like pizza
print(f"Your email is {email}")        # Outputs: Your email is bro123@fake.com

# 2. Integers (int)
age = 25
quantity = 3
num_of_students = 30

print(f"You are {age} years old")      # Outputs: You are 25 years old
print(f"You are buying {quantity} items") # Outputs: You are buying 3 items
print(f"Your class has {num_of_students} students")

# 3. Floating-Point Numbers (float)
price = 10.99
gpa = 3.2
distance = 5.5

print(f"The price is ${price}")        # Outputs: The price is $10.99
print(f"Your GPA is {gpa}")            # Outputs: Your GPA is 3.2
print(f"You ran {distance} km")        # Outputs: You ran 5.5 km

# 4. Booleans (bool)
is_student = True
for_sale = False
is_online = True

print(f"Are you a student? {is_student}") # Outputs: Are you a student? True
```

In production architectures, booleans are rarely displayed directly; instead, they dictate conditional control flow [10]:

```
if is_student:
    print("You are a student")
else:
    print("You are not a student")
```

## Type Casting

**Type casting** is the explicit process of converting a variable from one data type to another [12]. Python offers built-in constructor functions to transition data:

- `str()` : Converts data to a string [12].
- `int()` : Converts data to an integer (note: converting a float to an integer truncates and discards the decimal portion entirely; it does not round) [12, 13].
- `float()` : Converts data to a floating-point decimal [12].
- `bool()` : Converts data to a boolean [12, 14]. Any non-empty string or non-zero number typecasts to `True` [14]. **An empty string `""` or number `0` typecasts to `False`** [14, 15].

```
# Inspecting Data Types using type()
name = "Bro"
age = 25
gpa = 3.2
is_student = True

print(type(name))      # Outputs: <class 'str'>
print(type(age))       # Outputs: <class 'int'>
print(type(gpa))       # Outputs: <class 'float'>
print(type(is_student)) # Outputs: <class 'bool'>

# Executing Conversions
gpa_as_int = int(gpa)  # Converts 3.2 to 3 (truncation)
print(gpa_as_int)      # Outputs: 3

age_as_float = float(age)
print(age_as_float)    # Outputs: 25.0

# Illustrating String vs. Numeric operations
age_as_str = str(age)
# If we try age_as_str += 1, it raises a TypeError: can only concatenate str to str.
# If we concatenate a string "1":
age_as_str += "1"
print(age_as_str)      # Outputs: "251" (String concatenation, not addition!)

# Boolean Typecasting Rules
empty_string = ""
populated_string = "B"

print(bool(empty_string)) # Outputs: False
print(bool(populated_string)) # Outputs: True
```

## == vs. is : Equality vs. Identity

Coming from C/C++, `==` looks like the only comparison operator you need — but Python has two, and they answer different questions:

\* `==` asks: *"Do these two variables hold equal values?"* (calls the object's `__eq__` method — see Module 31).

\* `is` asks: *"Do these two variables point to the exact same object in memory?"* (the closest thing Python has to comparing raw pointer addresses).

```
list_a = [1, 2, 3]
list_b = [1, 2, 3] # a separate object with the same contents
list_c = list_a    # the SAME object as list_a

print(list_a == list_b) # True (same values)
print(list_a is list_b) # False (different objects in memory)
print(list_a is list_c) # True (literally the same object)
```

**Rule of thumb:** use `==` for comparing values (almost always what you want).

Use `is` only for checking against the singletons `None`, `True`, and `False` (e.g. `if x is None:` rather than `if x == None:` — this is the idiomatic, recommended style in Python).

## Module 2: User Input & Initial Projects

### Accepting User Input

The built-in `input()` function prompts the user to enter data in the console and **always returns the entry as a string ( `str` )** [15]. If you need to perform calculations or conditional checks on the input, you must typecast it [15, 18].

```
# Getting a name string
username = input("What is your name? ")
print(f"Hello {username}!")

# Getting an age integer (requires typecasting)
age_input = input("How old are you? ")
# age_input = int(age_input) # Step-by-step approach
# Alternatively, condense both steps into a single line:
age = int(input("How old are you? "))
print(f"You are {age} years old.")
```

## Chronological Course Projects: Input & Variables

## Project 1: Area of a Rectangle Calculator

This tool collects float measurements from the user, computes geometry, and prints the result utilizing Unicode superscript formatting.

```
# area_calculator.py
print("--- AREA OF A RECTANGLE CALCULATOR ---")

# Step 1: Collect user input and typecast directly to floats
length = float(input("Enter the length of the rectangle: "))
width = float(input("Enter the width of the rectangle: "))

# Step 2: Compute math
area = length * width

# Step 3: Print result with superscript 2 (cm²)
# To print the superscript '2' on Windows without copying/pasting:
# Ensure Num Lock is ON, hold ALT, and type '0178' on your numeric keypad.
print(f"The area is {area} cm²")
```

## Project 2: Simple Shopping Cart Program

This application models a retail transaction by managing items, floats for prices, and whole integer quantities, culminating in a formatted invoice.

```
# shopping_cart.py
print("--- SIMPLE SHOPPING CART PROGRAM ---")

# Step 1: Collect product variables
item = input("What item would you like to buy? ")
price = float(input("What is the price of the item? $"))
quantity = int(input("How many would you like to purchase? "))

# Step 2: Calculate total price
total_cost = price * quantity

# Step 3: Display invoice summary
print("\n" + "="*30)
print(f"You have bought {quantity} x {item}(s).")
print(f"Your total is: ${total_cost:.2f}")
print("="*30)
```

## Project 3: The Mad Libs Game

Mad Libs is an interactive word game where a custom story template is dynamically filled by accepting random categories of speech from user input [23].

```
# mad_libs.py
print("Welcome to Python Mad Libs!")
print("Fill in the blanks to create a funny story.\n")

# Step 1: Gather grammar variables
adjective_1 = input("Enter an adjective (description of something): ")
noun_1 = input("Enter a noun (person, place, or thing): ")
adjective_2 = input("Enter another adjective: ")
verb_1 = input("Enter a verb ending with 'ing' (action): ")
adjective_3 = input("Enter a final adjective: ")

# Step 2: Compile and display the customized story template
print("\n" + "-"*40)
print(f"Today I went to a {adjective_1} zoo.")
print(f"In an exhibit, I saw a {noun_1}.")
print(f"The {noun_1} was {adjective_2} and currently {verb_1}!")
print(f"I was extremely {adjective_3} to see it.")
print("-"*40)
```

---

## Module 3: Math and Circle Geometry Projects

### Arithmetic Operators

Python supports standard arithmetic, exponentiation, and remainder operators [30]:

- **Addition ( + )**: Sums two values [27].
- **Subtraction ( - )**: Deducts the right value from the left [28].
- **Multiplication ( \* )**: Multiplies values [28].

- **Division ( / )**: Divides left by right; **always returns a float** in Python [28].
- **Exponentiation ( \*\* )**: Raises a base number to a power [28].
- **Modulus ( % )**: Returns the division remainder [29]. (Crucial for checking parity: if `num % 2 == 0` the number is even; if it returns `1` it is odd) [29, 30].

## Augmented Assignment Operators

Shorthand operators that perform an arithmetic operation on a variable and then assign the result back to that same variable [27]:

```
friends = 5

friends += 1 # Equivalent to: friends = friends + 1 (Result: 6)
friends -= 2 # Equivalent to: friends = friends - 2 (Result: 4)
friends *= 3 # Equivalent to: friends = friends * 3 (Result: 12)
friends /= 2 # Equivalent to: friends = friends / 2 (Result: 6.0)
friends **= 2 # Equivalent to: friends = friends ** 2 (Result: 36.0)
friends %= 5 # Equivalent to: friends = friends % 5 (Result: 1.0)
```

## Built-in Math Functions

Python includes built-in functions for common math operations without requiring modules:

- \* `round(value, digits)` : Rounds a number to the nearest integer or specified decimal precision [30].
- \* `abs(value)` : Returns the absolute value (distance of a number from zero as a positive number) [30, 31].
- \* `pow(base, exp)` : Raises a base to the specified exponent [31].
- \* `max(x, y, z, ...)` : Evaluates multiple variables and returns the highest value [31].
- \* `min(x, y, z, ...)` : Evaluates and returns the lowest value [31].

```
x = 3.14
y = -4
z = 5

print(round(x)) # Outputs: 3
print(abs(y)) # Outputs: 4
print(pow(4, 3)) # Outputs: 64 (4 * 4 * 4)
print(max(x, y, z)) # Outputs: 5
print(min(x, y, z)) # Outputs: -4
```

## The `math` Module

For advanced calculations, import Python's standard `math` library [31]. It provides physical constants and specialized mathematical functions:

- \* **Constants:**
- \* `math.pi` : Mathematical constant  $\pi$  (approx. `3.14159...`) [31, 32].
- \* `math.e` : Euler's exponential constant (approx. `2.718...`) [32].
- \* **Functions:**
- \* `math.sqrt(x)` : Evaluates the square root of  $x$  [32].
- \* `math.ceil(x)` : Ceiling function; always rounds a float **up** to the nearest whole integer [32].
- \* `math.floor(x)` : Floor function; always rounds a float **down** to the nearest whole integer [32].

```
import math

print(math.pi) # Outputs: 3.141592653589793
print(math.e) # Outputs: 2.718281828459045
print(math.sqrt(9)) # Outputs: 3.0
print(math.ceil(9.1)) # Outputs: 10
print(math.floor(9.9)) # Outputs: 9
```

## Circle and Right Triangle Projects

### Project 4: Circumference of a Circle Calculator

Uses the circle formula  $C = 2\pi r$  to compute a circumference based on user radius input.

```
import math

print("--- CIRCUMFERENCE CALCULATOR ---")
radius = float(input("Enter the radius of the circle: "))

# C = 2 * pi * r
circumference = 2 * math.pi * radius

# Display rounded to 2 decimal places
print(f"The circumference is: {round(circumference, 2)} cm")
```

## Project 5: Area of a Circle Calculator

Uses the circle formula  $A = \pi r^2$  to compute area.

```
import math

print("--- AREA OF A CIRCLE CALCULATOR ---")
radius = float(input("Enter the radius of the circle: "))

# A = pi * r^2
area = math.pi * pow(radius, 2)

# Display rounded to 2 decimal places
print(f"The area of the circle is: {round(area, 2)} cm²")
```

## Project 6: Hypotenuse of a Right Triangle Calculator

Calculates the hypotenuse side  $C$  using the Pythagorean theorem:  $C = \sqrt{A^2 + B^2}$  [35].

```
import math

print("--- PYTHAGOREAN HYPOTENUSE CALCULATOR ---")
side_a = float(input("Enter the length of Side A: "))
side_b = float(input("Enter the length of Side B: "))

# C = sqrt(A^2 + B^2)
side_c = math.sqrt(pow(side_a, 2) + pow(side_b, 2))

print(f"Side C (the hypotenuse) is: {round(side_c, 2)}")
```

# Module 4: Control Flow, Decision-Making, & Calculator Projects

## Conditional Statements (If / Elif / Else)

Conditional control flow allows your program to execute different blocks of code based on whether specific logic evaluations are `True` or `False` [36].

- **Structure & Syntax:** Use `if`, followed by a condition, and end the line with a colon (`:`) [36]. The code block immediately below **must be indented** (typically 4 spaces) [36].
- **Comparison Operators:** Used to compare values [36].
  - `==` (Equality comparison; do not use single `=` which is for variable assignment!) [39]
  - `!=` (Inequality)
  - `>` (Greater than)
  - `<` (Less than)
  - `>=` (Greater than or equal to) [36]
  - `<=` (Less than or equal to)
- **Statement Execution:**
  - `if` : Evaluates a condition first [36].
  - `elif` (Else If): Evaluates secondary conditions chronologically if previous ones were False [37, 38].
  - `else` : Captures any remaining cases if all previous conditions were False [37].
- **Important Rule: Order of evaluation matters.** The interpreter runs from the top-down and executes *only* the first True code block, skipping the remaining branches [38]. Place more restrictive or specific checks (e.g., checking if age is  $\geq 100$ ) *before* broader checks (checking if age is  $\geq 18$ ) [38].

```
# Demonstrating basic conditional checks
age = int(input("Enter your age: "))

if age >= 100:
    print("You are too old to sign up!")
elif age < 0:
    print("You haven't been born yet!")
elif age >= 18:
    print("You are now signed up!")
else:
    print("You must be 18+ to sign up.")
```

## Control Flow Projects

### Project 7: Comprehensive Interactive Calculator

An on-the-fly arithmetic engine. It validates user operators and numerical inputs, performs correct math branches, handles decimal division rounding, and flags unhandled symbols.

```
# calculator.py
print("--- CHRONOLOGICAL CALCULATOR ---")

operator = input("Enter an operator (+ - * /): ")
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

if operator == "+":
    result = num1 + num2
    print(f"Result: {round(result, 3)}")
elif operator == "-":
    result = num1 - num2
    print(f"Result: {round(result, 3)}")
elif operator == "*":
    result = num1 * num2
    print(f"Result: {round(result, 3)}")
elif operator == "/":
    if num2 == 0:
        print("Error: Division by zero is mathematically undefined.")
    else:
        result = num1 / num2
        # Round the division result to 3 decimal places
        print(f"Result: {round(result, 3)}")
else:
    print(f"'{operator}' is not a valid arithmetic operator.")
```

### Project 8: Metric Weight Converter

Converts quantities between pounds ( **L** ) and kilograms ( **K** ), adjusting outputs dynamically. Includes safe invalid unit checks to protect operations.

```
# weight_converter.py
print("--- METRIC WEIGHT CONVERTER ---")

weight = float(input("Enter your weight: "))
unit = input("Is this weight in Kilograms or Pounds? (K/L): ").upper()

if unit == "K":
    # Convert Kilograms to Pounds
    converted_weight = weight * 2.205
    new_unit = "lbs"
    print(f"Your weight is {round(converted_weight, 1)} {new_unit}")
elif unit == "L":
    # Convert Pounds to Kilograms
    converted_weight = weight / 2.205
    new_unit = "kgs"
    print(f"Your weight is {round(converted_weight, 1)} {new_unit}")
else:
    print(f"Error: '{unit}' is an invalid unit selection. Use 'K' or 'L'.")
```

### Project 9: Temperature Conversion Program

Converts Fahrenheit to Celsius and vice-versa, outputting structured degrees strings.

```
# temp_converter.py
print("--- TEMPERATURE CONVERTER ---")

unit = input("Is the current temperature in Celsius or Fahrenheit? (C/F): ").upper()
temp = float(input("Enter the temperature value: "))

if unit == "C":
    # Formula: F = (9/5 * C) + 32
    converted_temp = (9 * temp / 5) + 32
    print(f"The temperature in Fahrenheit is {round(converted_temp, 1)}°F")
elif unit == "F":
    # Formula: C = (F - 32) * 5/9
    converted_temp = (temp - 32) * 5 / 9
    print(f"The temperature in Celsius is {round(converted_temp, 1)}°C")
else:
    print(f"Error: '{unit}' is an invalid unit selection.")
```

## Module 5: Logical Operators & Ternary Shortcuts

### Logical Operators

Logical operators link multiple conditions together to evaluate complex boolean expressions [49]:

1. **or**: Returns **True** if **at least one** condition is true [49]. The entire expression is evaluated as **True** [49].
2. **and**: Returns **True** only if **all** linked conditions are true [51].
3. **not**: Inverts a condition (converts **True** to **False** and vice-versa) [53, 54].

```
# 1. OR Example: Weather safety checks
temp = 25
is_raining = False

if temp > 35 or temp < 0 or is_raining:
    print("The outdoor event is cancelled.")
else:
    print("The outdoor event is still scheduled.")

# 2. AND Example: Hot & Sunny parameters
temp = 30
is_sunny = True

if temp >= 28 and is_sunny:
    print("It is hot and sunny outside!")

# 3. NOT Example: Inverting state
if not is_sunny:
    print("It is cloudy outside.")
```

### Simplifiable Chain Comparisons

Python allows you to write clean mathematical ranges instead of chaining multiple **and** expressions [52].

\* Clunky: `if temp < 28 and temp > 0:`

\* Simplified: `if 0 < temp < 28:` [52, 53]

### Conditional Expressions (Ternary Operators)

A **conditional expression** is a single-line shortcut for using an **if-else** statement [54]. It returns one of two values based on a condition [54].

\* **Formula:** `X if condition else Y` [54]

\* This returns `X` if the condition is **True**, otherwise it returns `Y` [54, 57].

```
num = 5
age = 13
user_role = "guest"

# 1. Direct print ternary evaluation
print("positive" if num > 0 else "negative")

# 2. Variable assignment based on parity
result = "even" if num % 2 == 0 else "odd"

# 3. Classifying age bounds
status = "adult" if age >= 18 else "child"

# 4. Access level check
access_level = "full access" if user_role == "admin" else "limited access"
```

## Module 6: String Methods & Username Input Validation

### Essential String Methods

Strings are immutable sequences of characters [58]. Python provides powerful methods to query or transform strings:

- `len(string)` : Built-in function returning the total count of characters, including spaces [58].
- `string.find(char)` : Returns the index position of the **first occurrence** of a character (starts at 0) [58, 59]. Returns `-1` if not found [59].
- `string.rfind(char)` : Reverse find; returns the index position of the **last occurrence** of a character [59].
- `string.capitalize()` : Capitalizes *only* the first character of the string and lowercases the rest [59, 60].
- `string.upper()` : Converts all characters to uppercase [60].
- `string.lower()` : Converts all characters to lowercase [60].
- `string.isdigit()` : Returns `True` if the string contains **only numeric digits** (no letters or spaces) [60, 61].
- `string.isalpha()` : Returns `True` if the string contains **only alphabetical letters** (returns `False` if there are spaces, symbols, or digits) [61].
- `string.count(char)` : Returns an integer counting the total occurrences of a specific character [61].
- `string.replace(old, new)` : Replaces all occurrences of a character/substring with a new one [62]. Can be used to strip spaces or dashes by replacing with an empty string `""` [62].

```
name = "Bro Code"
phone = "1-234-567-8901"

print(len(name))           # Outputs: 8
print(name.find("o"))      # Outputs: 2 (Index of first 'o')
print(name.rfind("o"))     # Outputs: 5 (Index of last 'o')
print(name.capitalize())   # Outputs: "Bro code"
print(name.upper())        # Outputs: "BRO CODE"
print(name.lower())        # Outputs: "bro code"
print(phone.count("-"))    # Outputs: 3
print(phone.replace("-", "")) # Outputs: "12345678901"
```

### Project 10: Username Input Validator

This program models interactive user sign-ups by checking input formatting rules [63].

```
# username_validator.py
print("--- USERNAME REGISTRATION VALIDATOR ---")

# Rules:
# 1. Username can't exceed 12 characters
# 2. Must not contain spaces
# 3. Must not contain numbers (alphabetical characters only)

username = input("Enter your desired username: ")

if len(username) > 12:
    print("Registration Error: Your username cannot be more than 12 characters.")
elif not username.find(" ") == -1:
    print("Registration Error: Your username cannot contain blank spaces.")
elif not username.isalpha():
    print("Registration Error: Your username cannot contain numbers or symbols.")
else:
    print(f"Registration Successful! Welcome, '{username}'.")
```

## Module 7: String Indexing, Slicing, & Credit Card Masking

### String Indexing & Slicing

Indexing is used to access individual elements of a sequence (like strings, lists, or tuples) [65]. Slicing is used to extract a subset of elements (a "slice") [65]. Both utilize the **indexing operator: square brackets** `[]` [65].

The slicing operator supports up to three fields separated by colons:

`[start : end : step]` [65]

- **start** : The index where the slice begins (inclusive) [65, 67]. Defaults to `0` [67].
- **end** : The index where the slice finishes (exclusive) [65, 67]. Defaults to the end of the string [68].
- **step** : The frequency of increment [65, 68]. A step of `2` selects every second character [69]. A negative step of `-1` reverses the sequence [70].

## Positive vs. Negative Indexing

Python supports negative indexing, which accesses elements relative to the end of the sequence [68]. `-1` represents the last character, `-2` the second-to-last, etc. [68].

```
credit_card = "1234-5678-9012-3456"

# 1. Retrieving specific characters
first_char = credit_card[0] # '1'
last_char = credit_card[-1] # '6'

# 2. Slicing with start and end fields (start is inclusive, end is exclusive)
first_group = credit_card[0:4] # "1234"
# Or omit 0:
first_group = credit_card[:4] # "1234" (Defaults to start of string)

# Slicing from index 5 to end of string:
remaining_digits = credit_card[5:] # "5678-9012-3456"

# 3. Stepping through strings
every_second_char = credit_card[::2] # Outputs: "13-6891-46"

# 4. Reversing strings using step of -1
reversed_card = credit_card[::-1] # Backward credit card string
```

## String Indexing Exercises

### Exercise 1: Credit Card Masking Program

Protects user privacy by masking all characters of a credit card string with `X` characters except for the final four digits [69].

```
# credit_mask.py
credit_number = "1234-5678-9012-3456"

# Slice the last 4 characters using negative indexing
last_four_digits = credit_number[-4:]

# Mask previous portions with Xs, then concatenate the raw slice
masked_number = "XXXX-XXXX-XXXX-" + last_four_digits

print(f"Card on File: {masked_number}") # Outputs: Card on File: XXXX-XXXX-XXXX-3456
```

### Exercise 2: Palindrome Check & String Reversal

Uses negative step slicing to reverse textual content.

```
word = input("Enter a word: ")
reversed_word = word[::-1]

print(f"Reversed: {reversed_word}")
if word.lower() == reversed_word.lower():
    print("This word is a palindrome!")
else:
    print("This is not a palindrome.")
```

## Module 8: Format Specifiers inside F-Strings

### Format Specifiers

Format specifiers are flags inside an F-string placeholder ( `{variable:flag}` ) that format numerical and textual values [70, 71].

Here are the most common format flags:

| Flag | Meaning                                                            | Example {val:flag} | Output    |
|------|--------------------------------------------------------------------|--------------------|-----------|
| :.Nf | Set decimal precision to \$N\$ floating-point places [72]          | {3.14159:.2f}      | 3.14      |
| :N   | Allocate minimum \$N\$ spaces to display the value [72]            | {10:5}             | 10        |
| :0N  | Zero-pad a number to fill a width of \$N\$ digits [72]             | {5:03}             | 005       |
| :<   | Left-justify the value within allocated spaces [72]                | {5:<10}            | 5         |
| :>   | Right-justify the value within allocated spaces [73]               | {5:>10}            | 5         |
| :^   | Center-align the value within allocated spaces [73]                | {5:^10}            | 5         |
| :+   | Precede positive values with a plus sign, negative with minus [73] | {42:+}             | +42       |
| :    | Precede positive values with a blank space (for alignment) [73]    | {42: }             | 42        |
| :,   | Format large numbers with a thousands separator comma [73]         | {1000000:,}        | 1,000,000 |

### Combining Flags

You can combine flags inside the format specifier to apply multiple formatting rules simultaneously (e.g., standardizing thousands, formatting precision, and displaying signs) [73, 74].

```
price_1 = 3000.14159
price_2 = -9870.65
price_3 = 1200.34

# Formats: positive sign representation, thousands separator, and 2-decimal floats
print(f"Price 1 is: {price_1:+,.2f}") # Outputs: Price 1 is: +3,000.14
print(f"Price 2 is: {price_2:+,.2f}") # Outputs: Price 2 is: -9,870.65
print(f"Price 3 is: {price_3:+,.2f}") # Outputs: Price 3 is: +1,200.34
```

## Module 9: While Loops & Compound Interest Calculator

### While Loops in Python

A **while loop** continually executes a block of code as long as its specified condition remains `True` [74].  
\* **Infinite Loops:** If the loop's condition never becomes `False` and there's no escape mechanism (like an update or `break`), your program runs indefinitely, consuming system resources [75]. Always design an **exit strategy** [75].

```
# 1. Simple validation loop (loops until user enters their name)
name = ""
while name == "":
    name = input("Enter your name: ")
print(f"Hello {name}!")

# 2. Numeric boundary validation loop
age = -1
while age < 0:
    print("Age can't be negative.")
    age = int(input("Enter your age: "))
print(f"You are {age} years old.")

# 3. Sentinel loop: pressing 'Q' to quit
food = ""
while not food == "Q":
    food = input("Enter a food you like (Q to quit): ").upper()
    if food != "Q":
        print(f"You like {food}!")
print("Goodbye!")
```

### The `break` Keyword

Inside any loop, the `break` statement immediately terminates execution of the loop and exits it entirely, bypassing any conditional evaluations [83, 84].

```
# Using while True with break for direct escaping
while True:
    number = int(input("Enter a number between 1 and 10: "))
    if 1 <= number <= 10:
        break # Escape the loop when input is valid
    else:
        print("Invalid number!")
```

## Project 11: Compound Interest Calculator

This application computes compound savings balances over time, incorporating strict input validation loops for the investment principal, interest rate, and years [79].

```
# compound_interest.py
print("--- COMPOUND INTEREST CALCULATOR ---")

# Step 1: Initialize values through validation loops using 'while True'
while True:
    principal = float(input("Enter the principal investment amount: $"))
    if principal > 0:
        break
    print("Validation Error: Principal must be greater than zero.")

while True:
    rate = float(input("Enter the interest rate (annual %): "))
    if rate >= 0:
        break
    print("Validation Error: Interest rate cannot be negative.")

while True:
    time = int(input("Enter the investment duration in years: "))
    if time > 0:
        break
    print("Validation Error: Investment time must be greater than zero.")

# Step 2: Calculate compound interest
# Mathematical formula: Total = Principal * (1 + rate / 100) ^ Time
total = principal * pow((1 + rate / 100), time)

# Step 3: Print result formatted with 2 decimal places
print("\n" + "="*40)
print(f"Calculated savings after {time} years:")
print(f"Final Balance: ${total:,.2f}")
print("="*40)
```

## Module 10: For Loops & Digital Countdown Timer

### For Loops in Python

A **for loop** executes a block of code a fixed number of times [84]. It iterates over an **iterable** sequence (like a numeric range, string, list, set, or dictionary) [84].

\* *When to use which:* Use a **while loop** for indeterminate iterations (such as waiting for user input) [88]. Use a **for loop** when the number of iterations is predetermined [84].

#### The `range()` Function

The `range(start, end, step)` function generates a sequence of integers:

\* `start` is inclusive (defaults to `0`) [85, 87].

\* `end` is **exclusive** [85, 87].

\* `step` is the count increment [86].

```
# 1. Counting from 1 to 10 (11 is exclusive)
for x in range(1, 11):
    print(x)

# 2. Counting backwards from 10 to 1
for x in reversed(range(1, 11)):
    print(x)

# 3. Stepping by twos (counting odd numbers)
for x in range(1, 11, 2):
    print(x) # 1, 3, 5, 7, 9

# 4. Iterating over a string sequence
credit_card = "1234-5678"
for character in credit_card:
    print(character)
```

#### Loop Control Keywords: `continue` & `break`

- **`continue`** : Skips the rest of the current iteration and jumps directly to the next cycle of the loop [87].
- **`break`** : Escapes the loop entirely [87, 88].

```
# 1. Skipping a specific iteration using 'continue'
for x in range(1, 11):
    if x == 13:
        continue # Skip 13
    print(x)

# 2. Early loop termination using 'break'
for x in range(1, 21):
    if x == 13:
        break # Exit the loop completely once we hit 13
    print(x)
```

## Project 12: Digital Countdown Timer

This program uses a backwards-stepping `for` loop, the standard system clock `time.sleep()`, mathematical modulus, and format specifiers to render a real-time digital clock countdown in the terminal [88, 90].

```
# countdown_timer.py
import time

print("--- DIGITAL COUNTDOWN TIMER ---")
total_seconds = int(input("Enter countdown time in seconds: "))

# Step 1: Count backwards from total_seconds down to 1
# Range parameters: start at total_seconds, end at 0, step by -1
for x in range(total_seconds, -1, -1):
    # Calculate hours, minutes, and remaining seconds from current value 'x'
    # 3600 seconds in an hour, 60 seconds in a minute
    hours = x // 3600
    minutes = (x // 60) % 60
    seconds = x % 60

    # Step 2: Output digital clock string using format padding flags
    # ':02' forces integers to pad with leading zeros to maintain 2 digits (e.g. 05:01:09)
    print(f"{hours:02}:{minutes:02}:{seconds:02}", end="\r")

    # Sleep the thread for exactly 1 second before next decrement
    time.sleep(1)

print("\nTIME'S UP!")
```

## Module 11: Nested Loops & Symbol Rectangle Printer

### Nested Loops in Python

A **nested loop** is a loop located inside the body of another loop [92].

\* **Outer loop**: Controls the larger cycles of execution [93, 94].

\* **Inner loop**: Completes all its iterations for **each individual cycle** of the outer loop [94, 95].

```
# Outer loop runs 3 times
for x in range(3):
    # Inner loop runs 3 times (total of 9 prints)
    for y in range(1, 4):
        print(y, end=" ")
    print() # Prints a blank newline between outer loops
```

### Custom Print End Character

By default, the `print()` function appends a newline character `\n` to the end of its output [93]. To suppress this behavior or substitute it with another character (e.g., spaces or separators), pass the `end` keyword argument [93]:

```
print("Hello", end=" ") # Appends a space instead of a newline
print("World!")         # Prints on the same line: "Hello World!"
```

## Project 13: Symbol Rectangle Printer

Constructs grids of customizable height and width out of specified symbols by combining user inputs, nested loops, and customized print endings [95].

```
# symbol_grid.py
print("--- SYMBOL RECTANGLE PRINTER ---")

# Step 1: Gather grid parameters
rows = int(input("Enter the number of rows: "))
columns = int(input("Enter the number of columns: "))
symbol = input("Enter a symbol to print the grid with: ")

# Step 2: Nested loops to construct geometry
# Outer loop is responsible for generating rows
for r in range(rows):
    # Inner loop is responsible for printing columns horizontally
    for c in range(columns):
        print(symbol, end="") # Print symbol on the same line
    print() # After finishing a row's columns, output a newline to step down
```

## Module 12: Core Collections & Interactive Shopping Cart

### Collections Introduction

Python collections are structured variables used to store multiple elements within a single variable name [97, 98]. The three core, general-purpose sequential collections are:

| Collection | Syntax Brackets                   | Ordering        | Mutability (Changeable)                                   | Duplicates Allowed                               | Speed & Use Case                                 |
|------------|-----------------------------------|-----------------|-----------------------------------------------------------|--------------------------------------------------|--------------------------------------------------|
| List       | Square <code>[]</code> [97]       | Ordered [102]   | Mutable (Changeable) [102]                                | Yes [102]                                        | Standard variable collection [108].              |
| Set        | Curly <code>{}</code> [104]       | Unordered [104] | Immutable (No updates to items; but can add/remove) [104] | No (Filters duplicates automatically) [104, 106] | Faster lookups via membership [106].             |
| Tuple      | Parentheses <code>()</code> [106] | Ordered [107]   | Immutable (Unchangeable) [107]                            | Yes [107]                                        | Faster than lists; protects constant data [107]. |

### Deep Dive: Lists

Lists maintain strict chronological index ordering [102]. Slicing, iteration, and membership operators are fully supported [98, 99, 101].

#### Important List Methods

```
fruits = ["apple", "orange", "banana", "coconut"]

fruits.append("pineapple") # Adds element to the end of the list [102]
fruits.remove("apple")     # Deletes specified element [103]
fruits.insert(0, "melon")  # Inserts value at a specified index [103]
fruits.sort()              # Sorts items alphabetically/numerically in-place [103]
fruits.reverse()           # Reverses current order of list items [103]
# To sort in reverse alphabetical order: call sort(), then reverse() [103]

print(fruits.index("banana")) # Returns index position of value [103]
print(fruits.count("banana")) # Counts occurrences of a duplicate value [104]
fruits.clear()                # Empties the list entirely [103]
```

### Deep Dive: Sets

Sets are unordered, meaning they cannot be accessed via index subscripts [104, 105]. Attempting `set_name[0]` raises a `TypeError` [105]. They automatically ignore duplicates [106].

```
colors = {"red", "green", "blue", "red"}
print(colors) # Outputs: {"red", "green", "blue"} (Duplicates auto-removed)

colors.add("yellow") # Adds element to set [106]
colors.remove("green") # Removes specific element [106]
colors.pop()          # Removes a random element from the set [106]
colors.clear()         # Clears the set [106]
```

### Deep Dive: Tuples

Tuples cannot be changed after creation (immutable) [107]. They are optimized for speed [107].

```
dimensions = (1920, 1080)
# dimensions[0] = 800 # Raises TypeError: tuple object does not support item assignment

print(dimensions.index(1080)) # Outputs: 1
print(dimensions.count(1920)) # Outputs: 1
```

## Copying Mutable Collections Safely (Shallow vs. Deep Copy)

Because lists, sets, and dicts are mutable and variables are references (Module 0, §3), writing `new_list = old_list` does **not** create a second list — it creates a second name pointing at the same list. To actually duplicate data:

```
original = [1, 2, 3]

# 1. Shallow copy — makes a new outer list, but nested objects are still shared
copy_a = original.copy()      # or: list(original), or original[:]
copy_a.append(4)
print(original) # [1, 2, 3] <- unaffected, good for flat lists

# 2. The shallow-copy trap with NESTED structures
nested = [[1, 2], [3, 4]]
shallow = nested.copy()
shallow[0].append(99)          # mutates the inner list, which IS shared
print(nested) # [[1, 2, 99], [3, 4]] <- original changed too!

# 3. Deep copy — recursively copies every nested object too
import copy
deep = copy.deepcopy(nested)
deep[0].append(100)
print(nested) # unaffected this time
```

**Rule of thumb:** `.copy()` is fine for flat lists/dicts/sets of immutable values (numbers, strings). The moment your collection contains other lists/dicts/objects, reach for `copy.deepcopy()` if you need true independence.

## Project 14: Interactive Shopping Cart Program

Uses sequential, mutable list structures ( `foods` and `prices` ) inside a continuous sentinel-controlled `while` loop [109]. It collects items, cast prices as floats, converts inputs to lowercase to gracefully process quit commands, outputs horizontal cart views, and formats totals [110, 111, 112].

```
# interactive_cart.py
print("--- SHOPPING CART PROGRAM ---")

# Step 1: Declare empty list collections and sum variable
foods = []
prices = []
total = 0.0

# Step 2: Data collection loop
while True:
    food = input("Enter a food item to purchase (Q to checkout): ")
    # Convert input to lowercase to validate exit signal gracefully
    if food.lower() == "q":
        break
    else:
        price = float(input(f"Enter the price of a {food}: $"))
        foods.append(food)
        prices.append(price)

# Step 3: Checkout and horizontal summary
print("\n" + "-"*5 + " YOUR CART " + "-"*5)
# Print items horizontally on a single line separated by spaces
for item in foods:
    print(item, end=" ")
print("\n" + "-"*21)

# Step 4: Calculate total price
for price in prices:
    total += price

# Step 5: Render invoice
print(f"Your total price is: ${total:.2f}")
```

## 2D Collections (Grids & Matrices)

A **two-dimensional collection** is a collection made up of other collections (such as lists of lists or tuples of tuples) [113, 117]. They are ideal for representing grids, coordinates, or tabular database structures [113].

- **Coordinates:** Access elements using two sets of square brackets `[row][column]` [115]. Rows and columns both start at index `0` [115].

```
# A 2D list of groceries consisting of 3 separate row lists
groceries = [
    ["apple", "orange", "banana", "coconut"], # Row 0
    ["celery", "carrots", "potatoes"],      # Row 1
    ["chicken", "fish", "turkey"]           # Row 2
]

# Accessing elements using row and column coordinates
print(groceries[0][0]) # Outputs: apple (Row 0, Col 0)
print(groceries[1][2]) # Outputs: potatoes (Row 1, Col 2)
print(groceries[2][1]) # Outputs: fish (Row 2, Col 1)

# Iterating over 2D structures requires nested loops:
for row in groceries:
    for item in row:
        print(item, end=" ")
    print() # Newline after each row
```

---

## Project 15: Telephone Keypad Generator

Models phone keypad layouts using 2D tuples [118]. It uses nested loops and formatted margins to align the rows into a clean numerical grid [118, 119].

```
# phone_keypad.py
print("--- TELEPHONE KEYPAD GRID ---")

# Step 1: Define the telephone grid structure using an immutable 2D tuple
keypad = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 9),
    ("*", 0, "#")
)

# Step 2: Nested loops to iterate and output coordinates
# Outer loop iterates through each row tuple
for row in keypad:
    # Inner loop iterates through individual keys in the current row
    for key in row:
        # Print keys horizontally with space separation
        print(key, end=" ")
    # Print a newline once a row is completed to start the next line
    print()
```

---

## Module 14: Interactive Science Quiz Game Project

### Project 16: Interactive Science Quiz Game

This multi-layered project combines tuples, list appends, loop tracking, index comparisons, and mathematical percentage typecasting to build an interactive terminal quiz game [119, 120].

```

# science_quiz.py
print("--- INTERACTIVE SCIENCE QUIZ GAME ---")

# Step 1: Define static data structures
# Immutable questions tuple
questions = (
    "How many elements are in the periodic table?: ",
    "Which animal lays the largest eggs?: ",
    "What is the most abundant gas in Earth's atmosphere?: ",
    "How many bones are in the human body?: ",
    "Which planet in the solar system is the hottest?: "
)

# 2D tuple storing corresponding options for each question
options = (
    ("A. 116", "B. 117", "C. 118", "D. 119"),
    ("A. Whale", "B. Crocodile", "C. Elephant", "D. Ostrich"),
    ("A. Nitrogen", "B. Oxygen", "C. Carbon Dioxide", "D. Hydrogen"),
    ("A. 206", "B. 207", "C. 208", "D. 209"),
    ("A. Mercury", "B. Venus", "C. Mars", "D. Jupiter")
)

# Tuple storing the correct alphabetical answers
answers = ("C", "D", "A", "A", "B")

# Mutable list to track user inputs
guesses = []

# Tracker variables
score = 0
question_num = 0

# Step 2: Run the quiz loop
for question in questions:
    print("***40)
    print(question)

    # Render corresponding choices from the 2D options tuple
    # We use the 'question_num' variable to index the correct options row
    for option in options[question_num]:
        print(option)

    user_guess = input("Enter your answer (A, B, C, or D): ").upper()
    guesses.append(user_guess)

    # Validate guess against the correct answer index
    if user_guess == answers[question_num]:
        score += 1
        print("CORRECT!")
    else:
        print(f"INCORRECT. The correct answer was {answers[question_num]}.")

    question_num += 1

# Step 3: Compile and display results
print("\n" + "="*15 + " QUIZ RESULTS " + "="*15)

print("Correct Answers: ", end="")
for answer in answers:
    print(answer, end=" ")
print()

print("Your Guesses:      ", end="")
for guess in guesses:
    print(guess, end=" ")
print()

# Calculate final percentage (convert decimal division to percentage integer)
final_score = int((score / len(questions)) * 100)
print(f"\nYour Final Score: {final_score}% ({score}/{len(questions)} correct)")
print("="*44)

```

## Module 15: Dictionaries & Movie Theater Concession Stand

### Dictionaries Deep Dive

A **dictionary** is an ordered, changeable collection of **key-value pairs** [124]. They act like phonebooks or indexes: you look up a unique **key** to retrieve its associated **value** [124, 125]. Keys must be unique, while values can repeat [124].

```

capitals = {
    "USA": "Washington DC",
    "India": "New Delhi",
    "China": "Beijing",
    "Russia": "Moscow"
}

# 1. Retrieving values safely using .get()
# If you look up a key using capitals["Japan"] and it doesn't exist, it raises a KeyError.
# Using get() returns 'None' instead, preventing crashes:
print(capitals.get("Japan")) # Outputs: None
print(capitals.get("USA"))   # Outputs: Washington DC

# 2. Modifying and updating dictionaries
# .update() inserts new key-value pairs or updates existing ones
capitals.update({"Germany": "Berlin"})
capitals.update({"USA": "Detroit"}) # Updates USA capital value

# 3. Removing elements
capitals.pop("China")      # Removes China key-value pair
capitals.popitem()         # Removes the most recently inserted key-value pair
# capitals.clear()         # Clears the dictionary entirely

```

## Iterating Over Dictionaries

```

# To extract all keys only (returns a list-like object)
print(capitals.keys())

# To extract all values only
print(capitals.values())

# To extract key-value pairs as tuples
print(capitals.items())

# Standard loop for keys:
for key in capitals:
    print(key)

# Clean loop for both keys and values using .items()
for key, value in capitals.items():
    print(f"Key: {key} -> Value: {value}")

```

## Project 17: Movie Theater Concession Stand

This project uses a dictionary to store menu items and prices [130]. It iterates over items to display the menu, uses input validation with `.get()` to filter out items not on the menu, and calculates the total [131, 133, 134].

```
# concession_stand.py
print("--- MOVIE CONCESSION STAND ---")

# Step 1: Define menu dictionary with string keys and float values
menu = {
    "pizza": 5.99,
    "nachos": 4.50,
    "popcorn": 6.00,
    "fries": 2.50,
    "chips": 1.75,
    "soda": 3.00,
    "lemonade": 3.25
}

cart = []
total = 0.0

# Step 2: Display the formatted menu to the user
print("-"*10 + " MENU " + "-"*10)
for item, price in menu.items():
    # Use formatting flags to align items (allocate 10 spaces) and format prices
    print(f"{item:10}: ${price:.2f}")
print("-"*26)

# Step 3: Purchase transaction loop
while True:
    food = input("Select an item to add to your cart (Q to checkout): ").lower()
    if food == "q":
        break
    # Safely query key presence using .get()
    elif menu.get(food) is not None:
        cart.append(food)
        print(f"Added {food} to your cart.")
    else:
        print("That item is not on our menu. Please try again.")

# Step 4: Calculate total and output cart details
print("\n" + "-"*10 + " CHECKOUT RECEIPT " + "-"*10)
for food in cart:
    total += menu.get(food)
    print(food, end=" ")
print()

print(f"Your final checkout total is: ${total:.2f}")
print("-"*38)
```

## Module 16: Random Module & Number Guessing Game

### The random Module

The `random` module provides utility functions for generating pseudo-random numbers and shuffling sequences [135]. To use it, add `import random` to the top of your program [135].

```
import random

# 1. Generating a random whole integer (randint)
# Parameters: low and high bounds (both are inclusive!)
dice_roll = random.randint(1, 6) # Generates random number from 1 to 6

# 2. Generating a random float (random)
# Generates a random floating-point decimal between 0 and 1
decimal_val = random.random()

# 3. Choosing a random item from a sequence (choice)
options = ("rock", "paper", "scissors")
computer_choice = random.choice(options) # Picks one option at random

# 4. Shuffling a mutable list in-place (shuffle)
deck = ["2", "3", "4", "Jack", "Queen", "King", "Ace"]
random.shuffle(deck) # Shuffles the list items in-place
print(deck)
```

### Project 18: Number Guessing Game

Implements a random target number game [138]. It uses `.isdigit()` to validate user input, validates range boundaries, tracks guesses, and exits once the target is guessed [141, 142].

```
# guess_the_number.py
import random

print("Welcome to the Python Number Guessing Game!")

lowest_num = 1
highest_num = 100
# Step 1: Pick a random integer target
answer = random.randint(lowest_num, highest_num)

guesses_count = 0
is_running = True

# Step 2: Main game loop
while is_running:
    guess = input(f"Guess a number between {lowest_num} and {highest_num}: ")

    # Validate input is a numeric digit before typecasting
    if not guess.isdigit():
        print("Invalid guess! Please enter a valid number.")
        continue

    # Convert validated string to an integer
    guess = int(guess)
    guesses_count += 1

    # Validate input boundary rules
    if guess < lowest_num or guess > highest_num:
        print("Out of Range! Please stay within the game boundaries.")
    elif guess < answer:
        print("Too Low! Try again.")
    elif guess > answer:
        print("Too High! Try again.")
    else:
        # Win condition reached
        print(f"\nCORRECT! The answer was {answer}.")
        print(f"It took you {guesses_count} guesses to win.")
        is_running = False # Set state to False to escape while loop
```

---

## Module 17: Rock, Paper, Scissors Game Project

---

### Project 19: Rock, Paper, Scissors Game

This project uses conditional checks and random choice evaluations [144, 146]. The while loop is controlled by a state variable ( `playing = True` ) instead of `while True` , which is a best practice for readability and refactoring [147].

```

# rock_paper_scissors.py
import random

print("--- ROCK, PAPER, SCISSORS GAME ---")

options = ("rock", "paper", "scissors")
playing = True

# Step 1: Main program state loop
while playing:
    player = None
    computer = random.choice(options)

    # Step 2: Player choice validation loop
    # Continues prompting as long as player input is not in choices
    while player not in options:
        player = input("Enter your choice (rock, paper, scissors): ").lower()
        if player not in options:
            print("Invalid selection! Please try again.")

    print(f"Player: {player}")
    print(f"Computer: {computer}")

    # Step 3: Process win/loss/tie outcomes
    if player == computer:
        print("It's a tie!")
    elif player == "rock" and computer == "scissors":
        print("You win! Rock breaks Scissors.")
    elif player == "paper" and computer == "rock":
        print("You win! Paper covers Rock.")
    elif player == "scissors" and computer == "paper":
        print("You win! Scissors cut Paper.")
    else:
        print("You lose! Better luck next time.")

    # Step 4: Ask to play again
    play_again = input("Do you want to play again? (y/n): ").lower()
    if play_again != "y":
        playing = False

print("\nThanks for playing!")

```

---

## Module 18: Dice Roller with ASCII Art Project

---

### Project 20: Dice Roller Program utilizing ASCII Art

This project rolls a customizable quantity of dice, calculates the total sum, and prints corresponding ASCII faces side-by-side using nested loops [152, 155].

```

# dice_roller.py
import random

print("--- CUSTOM DICE ROLLER ---")

# Step 1: Map dice numbers (1 to 6) to corresponding ASCII arts inside a dictionary
# Each value is a tuple representing the 5 rows of a die face
dice_art = {
    1: (
        "  _ _ _ _  ",
        "  |   |   |  ",
        "  |   ●   |  ",
        "  |   |   |  ",
        "  |___|___|  ",
    ),
    2: (
        "  _ _ _ _  ",
        "  | ●   |   |  ",
        "  |   |   |   |  ",
        "  |   |   | ●  |  ",
        "  |___|___|  ",
    ),
    3: (
        "  _ _ _ _  ",
        "  | ●   |   |  ",
        "  |   ●   |   |  ",
        "  |   |   | ●  |  ",
        "  |___|___|  ",
    ),
    4: (
        "  _ _ _ _  ",
        "  | ●   | ●   |  ",
        "  |   |   |   |  ",
        "  | ●   | ●   |  ",
        "  |___|___|  ",
    ),
    5: (
        "  _ _ _ _  ",
        "  | ●   | ●   |  ",
        "  |   ●   |   |  ",
        "  | ●   | ●   |  ",
        "  |___|___|  ",
    ),
    6: (
        "  _ _ _ _  ",
        "  | ●   | ●   |  ",
        "  | ●   | ●   |  ",
        "  | ●   | ●   |  ",
        "  |___|___|  ",
    )
}

# Step 2: Roll random values
num_of_dice = int(input("How many dice would you like to roll?: "))
rolled_values = []

for _ in range(num_of_dice):
    roll = random.randint(1, 6)
    rolled_values.append(roll)

# Calculate the sum total
total = sum(rolled_values)

# Step 3: Advanced side-by-side horizontal printing
# Each die face has exactly 5 horizontal rows (index 0 through 4)
# To print side-by-side, we must print Row 0 of all dice, then Row 1 of all dice, etc.
for line in range(5):
    for roll in rolled_values:
        # Retrieve the specific face from the dictionary, and get the specific row index
        print(dice_art.get(roll)[line], end=" ")
    print() # Newline once all dice have printed that row

print(f"\nRolled Values: {rolled_values}")
print(f"Total Combined Sum: {total}")

```

## Module 19: Functions, Parameter Rules, & Default/Keyword Args

### Writing Reusable Functions

A **function** is a block of reusable code that executes only when called (invoked) [156, 157]. You define functions using the `def`

keyword, followed by a unique name, parentheses, and a colon [157].

```
# 1. Defining a function
def greet_user():
    print("Hello!")

# 2. Calling/invoking the function
greet_user() # Executing the block once
```

## Parameters vs. Arguments

- **Arguments:** The actual data values you pass into a function when calling it [158].
- **Parameters:** The temporary variables defined in the function signature that receive these values [158].

```
# 'name' and 'age' are parameters
def celebrate_birthday(name, age):
    print(f"Happy Birthday to {name}!")
    print(f"You are {age} years old.")

# "Bro" and 25 are arguments
celebrate_birthday("Bro", 25)
```

## The `return` Statement

The `return` statement immediately terminates a function and sends a result back to the caller [161]. If a function does not have a `return` statement, it returns `None` by default.

```
def add(x, y):
    return x + y # Sends the sum back

result = add(10, 5) # Assigns returned value (15) to variable
print(result)
```

## Default Arguments

A **default argument** is a pre-assigned fallback value for a parameter [164]. If the argument is omitted when the function is called, the default value is used [164].

- **Placement Rule: Non-default (positional) arguments must precede default arguments** in the function signature [167, 168]. Placing a default parameter *before* a positional one raises a `SyntaxError` [167].

```
# Safe default tax parameters
# 'list_price' is positional; 'discount' and 'tax' have default fallbacks
def calculate_net_price(list_price, discount=0.0, tax=0.05):
    return list_price * (1 - discount) * (1 + tax)

# 1. Calling with only the positional argument (uses defaults)
print(calculate_net_price(500)) # Net price of 500, 0% disc, 5% tax ($525.00)

# 2. Overriding the discount default
print(calculate_net_price(500, 0.1)) # Net price of 500, 10% disc ($472.50)
```

## Exercise: Timer utilizing Sleep & Defaults

```
import time

def count_up(end, start=0):
    for x in range(start, end + 1):
        print(x)
        time.sleep(1)
    print("Done!")
```

## ⚠ The Mutable Default Argument Trap

This is the single most common Python gotcha for developers coming from any statically-typed language, because nothing about the syntax hints at the problem.

**Default argument values are evaluated only ONCE — when the function is defined, not each time it's called.** If that default is a mutable object (a list, dict, or set), every call that relies on the default **shares the same object**:

```
# WRONG – the same list object is reused across every call!
def add_item(item, cart=[]):
    cart.append(item)
    return cart

print(add_item("apple"))    # ['apple']
print(add_item("banana"))  # ['apple', 'banana'] <- surprise! not a fresh list
```

**Fix:** use `None` as the default, and create the fresh mutable object *inside* the function body:

```
# RIGHT
def add_item(item, cart=None):
    if cart is None:
        cart = []      # a brand-new list every single call
    cart.append(item)
    return cart

print(add_item("apple"))    # ['apple']
print(add_item("banana"))  # ['banana'] <- correct, independent list
```

## Keyword Arguments

A **keyword argument** is an argument preceded by its parameter name and an equals sign ( `parameter=value` ) [169, 170].

\* **Benefits:** Improves readability and allows you to pass arguments in **any order** [169].

\* **Rule:** If you mix positional and keyword arguments, **positional arguments must come first** [170].

```
def print_greeting(greeting, title, first, last):
    print(f"{greeting} {title} {first} {last}")

# Order doesn't matter when using keyword identifiers
print_greeting(title="Mr.", last="Squarepants", greeting="Hello", first="Spongebob")
```

# Module 20: Arbitrary Arguments (args, \*kwargs) & Shipping Label Generator

## Arbitrary Arguments

When you don't know how many arguments a user might pass into a function, Python uses unpacking operators [174]:

1. **\*args (Arbitrary Positional Arguments):** Prefixing a parameter with a single asterisk `*` packs any number of incoming positional arguments into an **immutable tuple** [174, 175].
2. **\*\*kwargs (Arbitrary Keyword Arguments):** Prefixing a parameter with double asterisks `**` packs any number of incoming keyword arguments into a **mutable dictionary** [177].

```
# 1. *args sum demonstration
# The function packs all positional parameters into a tuple named 'nums'
def sum_all_numbers(*nums):
    total = 0
    for num in nums:
        total += num
    return total

print(sum_all_numbers(1, 2, 3, 4, 5)) # Outputs: 15

# 2. **kwargs address demonstration
# Packs keyword arguments into a dictionary named 'details'
def print_user_details(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

print_user_details(username="Bro123", status="active", role="admin")
```

## Project 21: Custom Shipping Label Generator

This project combines arbitrary positional parameters ( `*args` ) to handle dynamic full-name structures and arbitrary keyword parameters ( `**kwargs` ) to unpack optional address information [179, 180]. It uses dictionary membership checks ( `in` ) and `.get()` to handle optional fields like apartments or PO Boxes [182, 183].

```
# shipping_label.py
def generate_shipping_label(*names, **address_info):
    print("--- SHIPPING LABEL ---")

    # Step 1: Print full-name parts by iterating through the names tuple
    for name in names:
        print(name, end=" ")
    print() # Newline after printing names

    # Step 2: Access the street (required field)
    # Using single quotes inside the .get() to prevent f-string bracket conflicts
    print(address_info.get('street'))

    # Step 3: Handle optional secondary fields using membership validations
    if 'apartment' in address_info:
        print(f"Apartment Number: {address_info.get('apartment')}")
    elif 'pobox' in address_info:
        print(f"PO Box: {address_info.get('pobox')}")

    # Step 4: Render city, state, and zip codes
    city = address_info.get('city')
    state = address_info.get('state')
    zip_code = address_info.get('zip')
    print(f"{city}, {state} {zip_code}")
    print("-" * 22)

# Calling with a complex name and standard apartment
generate_shipping_label("Dr.", "Spongebob", "Squarepants",
                        street="123 Fake Street",
                        apartment="100",
                        city="Detroit",
                        state="Michigan",
                        zip="48201")

# Calling with a PO Box and no apartment
generate_shipping_label("Joe", "Schmo",
                        street="456 Empty Road",
                        pobox="10001-A",
                        city="Dallas",
                        state="Texas",
                        zip="75001")
```

## Module 21: Iterables & Membership Operators

### Iterables in Python

An **iterable** is any object or collection that can return its elements one at a time [183, 184]. This makes them compatible with loops [184].

\* **Examples:** Strings, Lists, Tuples, Sets, and Dictionaries [188].

```
# Iterating over lists, tuples, or sets is straightforward
fruits = {"apple", "orange", "banana"}
for fruit in fruits:
    print(fruit)

# Iterating over strings returns characters
for char in "Python":
    print(char, end="-") # P-y-t-h-o-n-
```

### Iterating Over Dictionaries

```
grades = {"Sandy": "A", "Patrick": "D"}

# Default iteration returns keys only
for name in grades:
    print(name)

# .values() iterates over values only
for grade in grades.values():
    print(grade)

# .items() iterates over key-value pairs
for student, grade in grades.items():
    print(f"{student} scored {grade}")
```

### Membership Operators: `in` & `not in`

Membership operators evaluate sequences to test if a value is present [188]. They return a boolean `True` or `False` [189].

- `in` : Returns `True` if the value is found [189].
- `not in` : Returns `True` if the value is `not` found [189].

```
# 1. Checking characters in strings
email = "user@gmail.com"
if "@" in email and "." in email:
    print("This is a valid email formatting structure.")

# 2. Searching collections
students = {"Spongebob", "Patrick", "Sandy"}
search_name = "Squidward"

if search_name not in students:
    print(f"'{search_name}' was not found in the student registry.")
```

## `enumerate()` : Getting the Index AND the Value

Your C++ instinct for "I need the index too" is `for (int i = 0; i < n; i++)`. In Python, resist the urge to write `for i in range(len(items))` — the idiomatic tool is `enumerate()`, which yields `(index, value)` pairs directly:

```
fruits = ["apple", "banana", "cherry"]

# Clunky, C-style approach (avoid this in Python):
for i in range(len(fruits)):
    print(i, fruits[i])

# Idiomatic Python:
for index, fruit in enumerate(fruits):
    print(index, fruit)

# Optional: start counting from a different number
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}. {fruit}") # 1. apple 2. banana 3. cherry
```

## `zip()` : Iterating Multiple Sequences in Parallel

`zip()` pairs up elements from two or more iterables by position, stopping at the shortest one — this replaces manually indexing into parallel arrays (a very common C-style pattern):

```
names = ["Spongebob", "Patrick", "Sandy"]
scores = [95, 60, 100]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

## `Lambda` : Anonymous, Throwaway Functions

A `lambda` is a small, unnamed function defined inline — used when you need a quick function object (usually as an argument to something else) and defining a full `def` elsewhere would be overkill.

- **Syntax:** `lambda parameters: expression` (the expression's result is automatically returned — no `return` keyword, and no statements, only a single expression).

```
# A named function...
def square(x):
    return x ** 2

# ...vs. the equivalent lambda
square_lambda = lambda x: x ** 2
print(square_lambda(5)) # 25

# Lambdas shine as arguments to sorted(), map(), filter():
students = [("Sandy", 4.0), ("Spongebob", 3.2), ("Patrick", 2.1)]

# sorted() with a key= function: sort by the SECOND item in each tuple (the GPA)
by_gpa = sorted(students, key=lambda student: student[1], reverse=True)
print(by_gpa) # [('Sandy', 4.0), ('Spongebob', 3.2), ('Patrick', 2.1)]
```

## `sorted()`, `min()`, `max()` with `key=`

Any of these built-ins accept a `key=` function that controls what value is

compared, without altering the original data:

```
words = ["banana", "kiwi", "watermelon", "fig"]

print(sorted(words))           # alphabetical (default)
print(sorted(words, key=len))  # sorted by string length
print(max(words, key=len))     # 'watermelon' (the longest)
print(min(words, key=len))     # 'kiwi' (the shortest)
```

## Module 22: List Comprehensions & Match-Case Statements

### List Comprehensions

A **list comprehension** is a concise way to create lists in Python [193]. They are faster and more readable than traditional loops [193].

- **Formula:** `[expression for item in iterable if condition]` [193, 195]

```
# Standard double calculation using traditional loops:
doubles_clunky = []
for x in range(1, 6):
    doubles_clunky.append(x * 2)

# Clean, modern approach using a list comprehension:
doubles = [x * 2 for x in range(1, 6)] # Outputs: [2, 4, 6, 8, 10]

# Tripling values
triples = [y * 3 for y in range(1, 6)]

# Squaring values
squares = [z ** 2 for z in range(1, 6)]
```

### Filtering with Conditional Comprehensions

You can append conditional statements to filter elements during creation [195, 198].

```
numbers = [1, -2, 3, -4, 5, -6, 7]

# 1. Extract positive numbers only
positives = [num for num in numbers if num >= 0] # [1, 3, 5, 7]

# 2. Extract negative numbers only
negatives = [num for num in numbers if num < 0]  # [-2, -4, -6]

# 3. Filter even numbers
evens = [num for num in numbers if num % 2 == 0]

# 4. Filter odd numbers
odds = [num for num in numbers if num % 2 != 0]

# 5. Extract passing scores (>= 60)
scores = [85, 42, 79, 90, 56, 61, 30]
passing = [score for score in scores if score >= 60] # [85, 79, 90, 61]
```

### Dictionary & Set Comprehensions

The same `[... for ... in ...]` pattern extends to dictionaries and sets using `{}` instead of `[]` — very handy, and easy to miss if you only ever hear about "list comprehensions":

```
# Dictionary comprehension: {key_expr: value_expr for item in iterable}
names = ["Spongebob", "Patrick", "Sandy"]
name_lengths = {name: len(name) for name in names}
print(name_lengths) # {'Spongebob': 9, 'Patrick': 7, 'Sandy': 5}

# Set comprehension: {expr for item in iterable} — auto-dedupes like any set
numbers = [1, 2, 2, 3, 3, 3, 4]
unique_squares = {n ** 2 for n in numbers}
print(unique_squares) # {1, 4, 9, 16}
```

### Match-Case Statements

Introduced in modern Python, **match-case statements** are clean, highly structured alternatives to using clunky `if-elif-else` chains [200]. If you've used other programming languages, this is Python's version of a **switch** statement [200].

- **Syntax:** Use `match`, followed by the variable to examine [201]. Indent multiple `case` statements below [201].

- **Wildcard `case _`**: The underscore acts as a fallback case, matching any remaining inputs (behaves like an `else` statement) [201, 202].
- **Grouping Cases**: Combine multiple cases on a single line using the pipe operator `|` to represent logical `or` [203].

```
# 1. Standard Case Mapping
def get_day_name(day_num):
    match day_num:
        case 1:
            return "Sunday"
        case 2:
            return "Monday"
        case 3:
            return "Tuesday"
        case 4:
            return "Wednesday"
        case 5:
            return "Thursday"
        case 6:
            return "Friday"
        case 7:
            return "Saturday"
        case _:
            return "Invalid Day Number" # Wildcard fallback

# 2. Chaining Cases using pipe operator '|'
def check_weekend(day_name):
    match day_name.lower():
        case "saturday" | "sunday":
            return True
        case "monday" | "tuesday" | "wednesday" | "thursday" | "friday":
            return False
        case _:
            return "Invalid day string!"
```

## Module 23: Modules, Scope Resolution (LEGB), & the Entry Point

### Modules in Python

A **module** is simply a Python file ( `.py` ) containing variable declarations and functions that you want to include in another program [204, 207]. Modules promote separation of concerns and code reuse [204].

### Standard Import Syntaxes

```
# 1. Standard import (calls must be prefixed with module name)
import math
print(math.pi)

# 2. Aliasing imports (sets a custom nickname to reduce clunky syntax)
import math as m
print(m.pi)

# 3. Direct imports
from math import pi
print(pi) # Imports directly into namespace; no prefix needed
```

### Creating Custom Modules

You can easily create your own module files and import them:

```
# example_module.py (Saved in same directory)
pi_value = 3.14159

def calculate_area(radius):
    return pi_value * radius ** 2
```

To use this custom file in your main script:

```
# main.py
import example_module

# Accessing module variables and functions:
result = example_module.calculate_area(5)
print(result)
```

### Variable Scope & LEGB Scope Resolution

**Variable scope** refers to the regions of a program where a specific variable is visible and accessible [209, 210].

Python resolves variable name conflicts using the **LEGB rule** [209]:

1. **L (Local)**: Variables declared inside the current function [209]. They are visible *only* inside that function [209].
2. **E (Enclosed)**: Variables declared inside an outer, enclosing function (when functions are nested) [211].
3. **G (Global)**: Variables declared outside any function in the module file [211, 212].
4. **B (Built-in)**: Core keywords and constants built into Python itself (e.g., `print()`, `abs()`, `math.e`) [212].

```
# Global variable
x = "Global Variable"

def outer_function():
    # Enclosed variable
    x = "Enclosed Variable"

    def inner_function():
        # Local variable
        x = "Local Variable"
        print(x) # Accesses Local first!

    inner_function()

outer_function()
```

## Modifying Outer-Scope Variables: `global` and `nonlocal`

LEGB explains where Python *looks up* a name — but by default, assigning to a name inside a function always creates a new **local** variable, even if a global variable with the same name already exists. This surprises C/C++ developers because in C++, an inner scope naturally sees and can mutate an outer variable unless you explicitly shadow it.

```
counter = 0

def increment_broken():
    counter += 1 # UnboundLocalError! Python sees the assignment and
                # decides 'counter' is local for the WHOLE function body,
                # so reading it before assigning fails.

def increment_fixed():
    global counter # explicitly tells Python: use the module-level 'counter'
    counter += 1

increment_fixed()
print(counter) # 1
```

`nonlocal` is the equivalent tool for nested functions, letting an inner function modify a variable from its *enclosing* (not global) scope:

```
def make_counter():
    count = 0
    def increment():
        nonlocal count # refers to 'count' in make_counter(), not a new local
        count += 1
        return count
    return increment

counter = make_counter()
print(counter()) # 1
print(counter()) # 2
```

**Rule of thumb:** reaching for `global` / `nonlocal` a lot is usually a sign you should restructure the code (e.g. return values instead, or use a class) — but knowing they exist prevents a lot of confused debugging.

## The Main Entry Point

You will often see this condition at the bottom of Python files:

```
if __name__ == '__main__': [213, 214]
```

- **How it works:**
- Python automatically assigns a special internal variable `__name__` to every script at runtime [216].
- If a file is executed **directly** as the main program, Python sets `__name__` to `'__main__'` [216].
- If a file is **imported** into another script as a module, Python sets `__name__` to the file's name [216].
- **Why it matters:** Wrapping your driver code inside this conditional prevents code from executing automatically when another script imports functions from your file [214, 218].

```
# script_one.py
def utility_function():
    print("Utility operation.")

# Only execute this block if run directly
if __name__ == '__main__':
    print("Running directly as a standalone program!")
    utility_function()
```

---

## Module 24: Robust CLI ATM Banking Project

---

### Project 22: Robust CLI ATM Banking Program

A production-ready CLI banking dashboard. It encapsulates modular functions to manage bank account balances, processes deposits, executes withdrawals, checks for overdraft limits, validates inputs, and uses the `if __name__ == '__main__':` entry point pattern [220, 221, 228].

```

# atm_banking.py

# Function 1: Display the current formatted balance
def show_balance(current_balance):
    print(""" * 30)
    print(f"Your Account Balance is: ${current_balance:,.2f}")
    print(""" * 30)

# Function 2: Deposit transaction with verification checks
def deposit():
    print(""" * 30)
    amount_str = input("Enter the amount you would like to deposit: $")
    # Validate the input is numeric
    try:
        amount = float(amount_str)
    except ValueError:
        print("Error: Input is not a valid number.")
        return 0.0

    if amount <= 0:
        print("Error: Deposit amount must be greater than zero.")
        return 0.0

    print(f"Success! Deposited: ${amount:,.2f}")
    return amount

# Function 3: Withdrawal transaction with verification checks
def withdraw(current_balance):
    print(""" * 30)
    amount_str = input("Enter the amount to withdraw: $")
    try:
        amount = float(amount_str)
    except ValueError:
        print("Error: Input is not a valid number.")
        return 0.0

    if amount <= 0:
        print("Error: Withdrawal amount must be greater than zero.")
        return 0.0
    elif amount > current_balance:
        print("Error: Insufficient funds! Overdraft declined.")
        return 0.0

    print(f"Success! Withdrew: ${amount:,.2f}")
    return amount

# Function 4: Main banking controller
def main():
    balance = 0.0
    is_running = True

    while is_running:
        print("\n" + "=" * 10 + " PYTHON ATM BANKING " + "=" * 10)
        print("1. Show Balance")
        print("2. Deposit Money")
        print("3. Withdraw Money")
        print("4. Exit Program")

        choice = input("Enter your choice (1-4): ")

        if choice == "1":
            show_balance(balance)
        elif choice == "2":
            deposit_amount = deposit()
            balance += deposit_amount
        elif choice == "3":
            withdraw_amount = withdraw(balance)
            balance -= withdraw_amount
        elif choice == "4":
            is_running = False
            print("Session complete.")
        else:
            print("Invalid Choice! Please enter a number between 1 and 4.")

    print("\nThank you for banking with us. Have a wonderful day!")

# Safe import execution entry point
if __name__ == '__main__':
    main()

```

## Project 23: Classic Slot Machine Simulator

This game uses list comprehensions to spin reels populated with custom emojis [231, 236]. It prints reels vertically aligned with pipes `|` [237]. If all three reels match, the program evaluates the win types and adds cash back to the user's balance [238, 240].

```
# slot_machine.py
import random

# Global Constants: Emojis and corresponding win multipliers
# Cherry = x3, Watermelon = x4, Lemon = x5, Bell = x10, Star = x20
EMOJIS = ("🍒", "🍉", "🍋", "🔔", "★")

def spin_reel():
    # Return 3 random symbols using a list comprehension
    # The underscore '_' acts as a standard loop variable placeholder
    return [random.choice(EMOJIS) for _ in range(3)]

def print_reels(reel):
    print("*" * 15)
    # Join reel lists using spacer pipes
    print(" | ".join(reel))
    print("*" * 15)

def calculate_payout(reel, bet):
    # Check if all three elements match
    if reel[0] == reel[1] == reel[2]:
        symbol = reel[0]
        if symbol == "🍒":
            multiplier = 3
        elif symbol == "🍉":
            multiplier = 4
        elif symbol == "🍋":
            multiplier = 5
        elif symbol == "🔔":
            multiplier = 10
        elif symbol == "★":
            multiplier = 20

        winnings = bet * multiplier
        return winnings
    return 0

def main():
    balance = 100.0
    print("=" * 10 + " PYTHON SLOT MACHINE " + "=" * 10)
    print("Symbols: 🍒 (x3) | 🍉 (x4) | 🍋 (x5) | 🔔 (x10) | ★ (x20)")

    while balance > 0:
        print(f"\nCurrent Balance: ${balance:.2f}")
        bet_input = input("Place your bet amount: $")

        # Validation checks
        if not bet_input.isdigit():
            print("Error: Please enter a valid whole number.")
            continue

        bet = int(bet_input)

        if bet <= 0:
            print("Error: Bet must be greater than zero.")
            continue
        elif bet > balance:
            print("Error: Insufficient funds!")
            continue

        # Deduct bet amount
        balance -= bet
        print("Spinning...")

        # Spin and show reels
        reel = spin_reel()
        print_reels(reel)

        # Compute payouts
        winnings = calculate_payout(reel, bet)

        if winnings > 0:
            balance += winnings
            print(f"YOU WON: ${winnings:.2f}!")
        else:
            print("Sorry, you lost this round.")

        # Ask to continue
```

```

        play_again = input("Do you want to spin again? (y/n): ").upper()
        if play_again != "Y":
            break

    print(f"\nGAME OVER. Your final balance is: ${balance:.2f}")
    print("Thanks for playing!")

if __name__ == '__main__':
    main()

```

## Module 26: Substitution Cipher Encryption System Project

### Project 24: Substitution Cipher Encryption & Decryption System

A complete introductory cryptography tool [243]. It imports string class character libraries to build the alphabet pool, copies and shuffles characters using a random key array, and maps indices to secure plaintext messages before decrypting them [243, 244, 245, 248].

```

# substitution_cipher.py
import random
import string

# Step 1: Create characters pool by combining string module constants
# We include ascii letters, decimal digits, punctuations, and a blank space
CHAR_POOL = string.ascii_letters + string.digits + string.punctuation + " "

# Cast character pool to a list structure
chars = list(CHAR_POOL)

# Copy lists and shuffle to establish a key
key = chars.copy()
random.shuffle(key)

# Display encryption characters for reference (hidden from final outputs)
# print(f"Alphabet: {chars}")
# print(f"Key:      {key}")

def encrypt(plain_text):
    cipher_text = ""
    # Iterate through characters of the message
    for letter in plain_text:
        # Locate index position of letter in base alphabet
        if letter in chars:
            idx = chars.index(letter)
            # Retrieve substituted character at that same index in key
            cipher_text += key[idx]
        else:
            # Keep character raw if not found in pool
            cipher_text += letter
    return cipher_text

def decrypt(cipher_text):
    plain_text = ""
    # Reverse index mapping: lookup in key first, map back to base alphabet
    for letter in cipher_text:
        if letter in key:
            idx = key.index(letter)
            plain_text += chars[idx]
        else:
            plain_text += letter
    return plain_text

def main():
    print("--- SUBSTITUTION CIPHER SYSTEM ---")
    message = input("Enter a message to encrypt: ")

    # Run Encryption
    encrypted_msg = encrypt(message)
    print(f"\nOriginal Message:  {message}")
    print(f"Encrypted Cipher:  {encrypted_msg}")

    # Run Decryption
    decrypted_msg = decrypt(encrypted_msg)
    print(f"Decrypted Message: {decrypted_msg}")

if __name__ == '__main__':
    main()

```

# Module 27: Hangman Game Project with Gallows ASCII Art

## Project 25: Hangman Game

This project implements the classic game of Hangman [249]. It leverages a separate module for its word database [265], implements a dictionary to represent different states of the gallows [250], validates user input [261], and manages guessed letters using sets [262].

```
# words_list.py
# (Create this file in the same directory as hangman.py)
words = ("apple", "orange", "banana", "coconut", "pineapple", "watermelon", "grape")
```

```
# hangman.py
import random
from words_list import words

# Map incorrect guesses count to gallows ASCII visual states
hangman_art = {
    0: (
        "  ",
        "  ",
        "  "
    ),
    1: (
        " 0 ",
        "  ",
        "  "
    ),
    2: (
        " 0 ",
        " | ",
        "  "
    ),
    3: (
        " 0 ",
        " /| ",
        "  "
    ),
    4: (
        " 0 ",
        " /|\ ", # Escape sequence double backslash to display singular backslash
        "  "
    ),
    5: (
        " 0 ",
        " /|\ ",
        " /  "
    ),
    6: (
        " 0 ",
        " /|\ ",
        " / \ "
    )
}

def display_man(wrong_guesses):
    print("*" * 30)
    # Loop and print each row of the current gallows tuple
    for line in hangman_art[wrong_guesses]:
        print(line)
    print("*" * 30)

def display_hint(hint):
    # Join spaces between the blanks list for easier reading
    print(" ".join(hint))

def display_answer(answer):
    print(f"The correct word was: {answer}")

def main():
    print("---- WELCOME TO PYTHON HANGMAN ----")

    answer = random.choice(words)
    # Create matching placeholders list
    hint = ["_"] * len(answer)

    wrong_guesses = 0
    guessed_letters = set() # Store player inputs using set structures
    is_running = True

    while is_running:
        display_man(wrong_guesses)
        display_hint(hint)

        guess = input("\nEnter your guess (single letter): ").lower()

        # 1. Validation: single letter only
        if len(guess) != 1:
            print("Invalid Input! You can only guess one letter at a time.")
            continue
        # 2. Validation: must be alphabetical character
        if not guess.isalpha():
            print("Invalid Input! Numbers and symbols are not allowed.")
            continue
        # 3. Validation: prevent duplicate guesses
        if guess in guessed_letters:
```

```

        print(f"You have already guessed '{guess}'. Please try a different letter.")
        continue

# Add validated input to tracking set
guessed_letters.add(guess)

# Search the word for matches
if guess in answer:
    # Map occurrences and replace hint blanks with the guess
    for idx in range(len(answer)):
        if answer[idx] == guess:
            hint[idx] = guess
else:
    wrong_guesses += 1
    print("Incorrect guess!")

# Check Win state
if "_" not in hint:
    display_man(wrong_guesses)
    display_answer(answer)
    print("\nCongratulations! YOU WIN!")
    is_running = False

# Check Loss state
if wrong_guesses >= 6:
    display_man(wrong_guesses)
    display_answer(answer)
    print("\nGame Over! You have run out of guesses.")
    is_running = False

if __name__ == '__main__':
    main()

```

## Module 28: Object-Oriented Programming (OOP) Deep Dive

### OOP Introduction

**Object-Oriented Programming (OOP)** is a design paradigm that structures code into reusable units called **objects** [266, 267]. An object is an encapsulated bundle of:

1. **Attributes:** Similar to variables; they define what an object **has** (description states) [266].
2. **Methods:** Similar to functions; they define what an object **does** (actions) [266, 267].

### Classes

A **class** is a blueprint or template used to define the state structures (attributes) and behavior rules (methods) that objects instantiated from it will possess [267].

```

# car.py (Class blueprint file)
class Car:
    # 1. Constructor Method: __init__ (dunder init)
    # This dunder method is automatically invoked when constructing new objects
    # 'self' is a parameter representing the current object instance being created
    def __init__(self, model, year, color, for_sale):
        self.model = model      # Instance attribute
        self.year = year        # Instance attribute
        self.color = color      # Instance attribute
        self.for_sale = for_sale # Instance attribute

    # 2. Instance Methods
    # Every instance method must accept 'self' as its first parameter
    def drive(self):
        print(f"You drive the {self.color} {self.model}.")

    def stop(self):
        print(f"You stop the {self.color} {self.model}.")

    def describe(self):
        print(f"This is a {self.year} {self.color} {self.model}.")

```

### Instantiating Objects

To construct objects from your class blueprint, import the file and call the class name as if it were a function [269, 271]. Python automatically manages the `self` parameter behind the scenes [269].

```
# main.py
from car import Car

# Instantiate three unique car objects
car_1 = Car("Mustang", 2024, "red", False)
car_2 = Car("Corvette", 2025, "blue", True)
car_3 = Car("Charger", 2026, "yellow", True)

# Accessing attributes using the attribute access operator (dot '.')
print(car_1.model) # Outputs: Mustang
print(car_2.color) # Outputs: blue

# Invoking instance methods
car_1.drive()      # Outputs: You drive the red Mustang.
car_2.stop()       # Outputs: You stop the blue Corvette.
car_3.describe()   # Outputs: This is a 2026 yellow Charger.
```

## Class Variables vs. Instance Variables

- **Instance Variables:** Declared *inside* the class constructor ( `__init__` ) [274]. Each object has its own unique copy of these variables [276].
- **Class Variables:** Declared *outside* the constructor directly in the class body [274, 276]. They are shared among **all** object instances created from the class [274].
- **Best Practice:** Access class variables using the **Class Name** directly instead of object instances to improve readability and indicate class-level scope [276, 277].

```
class Student:
    # Class Variables (shared among all student objects)
    graduating_class = 2025
    num_students = 0

    def __init__(self, name, age):
        # Instance Variables (unique to each student object)
        self.name = name
        self.age = age
        # Increment class variable whenever a new student is instantiated
        Student.num_students += 1

student_1 = Student("Spongebob", 30)
student_2 = Student("Patrick", 35)

# Accessing instance variables (unique)
print(student_1.name) # Spongebob
print(student_2.name) # Patrick

# Accessing class variables (shared)
print(Student.graduating_class) # Outputs: 2025
print(Student.num_students)     # Outputs: 2
```

## Module 29: Advanced Inheritance & Super() Shape Exercises

### Inheritance in Python

**Inheritance** allows a child class to inherit attributes and methods from a parent class [280]. This reduces redundancy and makes your code easier to maintain [280, 283].

- **Syntax:** Define the child class and pass the parent class name in parentheses [281].

```
# Parent Class (Superclass)
class Animal:
    def __init__(self, name):
        self.name = name
        self.is_alive = True

    def sleep(self):
        print(f"{self.name} is sleeping...")

# Child Class (Subclass) inherits from Animal
class Dog(Animal):
    def speak(self):
        print(f"{self.name} says: Woof!")

dog_1 = Dog("Scooby")
dog_1.sleep() # Inherited method (Outputs: Scooby is sleeping...)
dog_1.speak() # Child-specific method (Outputs: Scooby says: Woof!)
```

## Multiple vs. Multi-Level Inheritance

1. **Multiple Inheritance:** A child class inherits directly from **multiple** parent classes [285].  
\* *Syntax:* `class Fish(Prey, Predator):` [286]
2. **Multi-Level Inheritance:** A chain of inheritance where a child inherits from a parent, which in turn inherits from its own parent (grandparent relationship) [285, 287].  
\* *Example:* `Animal` (Grandparent) → `Predator` (Parent) → `Hawk` (Child) [287].

```
# 1. Multiple Inheritance Setup
class Prey:
    def flee(self):
        print("This animal is fleeing.")

class Predator:
    def hunt(self):
        print("This animal is hunting.")

# Fish inherits from both Prey and Predator
class Fish(Prey, Predator):
    pass

nemo = Fish()
nemo.flee() # Outputs: This animal is fleeing.
nemo.hunt() # Outputs: This animal is hunting.
```

## The `super()` Function

The `super()` function allows a child class to invoke methods (most commonly the constructor `__init__`) from its parent class [290]. This allows you to delegate shared attribute assignments to the parent class, avoiding code duplication [292, 293].

## Method Overriding

If a child class defines a method with the same name as a method in its parent class, the child's method **overrides** the parent's version [297].

---

## Object-Oriented Geometry Exercises

### Shape Inheritance Exercise

This program implements geometric shapes (Circle, Square, Triangle) inheriting from a base `Shape` class [290, 291]. It uses `super()` to handle shared attributes (color and fill state) and uses method overriding to calculate area for each shape [293, 296, 297].

```
# shape_hierarchy.py
class Shape:
    def __init__(self, color, is_filled):
        self.color = color
        self.is_filled = is_filled

    def describe(self):
        fill_text = "filled" if self.is_filled else "empty"
        print(f"This shape is colored {self.color} and is {fill_text}.")

class Circle(Shape):
    def __init__(self, color, is_filled, radius):
        # Call parent constructor using super() to assign shared attributes
        super().__init__(color, is_filled)
        self.radius = radius

    # Method Overriding: override parent 'describe' to add circle details
    def describe(self):
        super().describe() # Optionally invoke the parent's method logic first!
        area = 3.14159 * (self.radius ** 2)
        print(f"It is a Circle with a radius of {self.radius} cm and Area of {area:.2f} cm².")

class Square(Shape):
    def __init__(self, color, is_filled, width):
        super().__init__(color, is_filled)
        self.width = width

    def describe(self):
        super().describe()
        area = self.width ** 2
        print(f"It is a Square with a width of {self.width} cm and Area of {area:.2f} cm².")

class Triangle(Shape):
    def __init__(self, color, is_filled, width, height):
        super().__init__(color, is_filled)
        self.width = width
        self.height = height

    def describe(self):
        super().describe()
        area = 0.5 * self.width * self.height
        print(f"It is a Triangle with base {self.width} cm, height {self.height} cm, and Area of {area:.2f} cm².")

# Instantiate shapes
circle = Circle("red", True, 5)
square = Square("blue", False, 6)
triangle = Triangle("yellow", True, 7, 8)

circle.describe()
square.describe()
triangle.describe()
```

## Module 30: Polymorphism, Duck Typing, & Static/Class Methods

### Polymorphism

**Polymorphism** (from the Greek meaning "many forms") is a programming concept where different objects can be treated as instances of a shared parent class [299]. This allows you to call the same method name on different objects, and each object will execute its own overridden implementation [299, 302].

```
# Utilizing the Shape classes from our previous module:
shapes = [
    Circle("red", True, 5),
    Square("blue", False, 6),
    Triangle("yellow", True, 7, 8)
]

# Iterate through shapes list polymorphism in action
for shape in shapes:
    # Every shape object executes its own custom describe() method!
    shape.describe()
print("-" * 30)
```

### Duck Typing

**Duck Typing** is a form of dynamic typing where an object's suitability is determined by the presence of specific methods and attributes, rather than its inheritance hierarchy [304].

\* *Adage*: "If it looks like a duck and quacks like a duck, it must be a duck." [305]

\* In Python, you can treat completely unrelated classes the same way as long as they implement the **same method names** [304, 305].

```
class Dog:
    def speak(self):
        print("Woof!")

class Cat:
    def speak(self):
        print("Meow!")

class Car:
    # Car is not an Animal, but implements speak() to sound its horn
    def speak(self):
        print("Honk!")

# Dynamic function accepting any object with a 'speak' method
def make_it_speak(entity):
    entity.speak()

make_it_speak(Dog()) # Outputs: Woof!
make_it_speak(Car()) # Outputs: Honk! (Works via Duck Typing!)
```

## Static Methods ( @staticmethod )

A **static method** is a method that belongs to a class rather than any individual object instantiated from it [307].

\* **Characteristics:** They do **not** accept `self` or `cls` as their first parameter [309]. They behave like standalone utility functions scoped inside a class [308, 309].

\* **Decorator:** Defined using the `@staticmethod` decorator [309].

```
class Employee:
    def __init__(self, name, position):
        self.name = name
        self.position = position

    # Instance Method (requires self)
    def get_info(self):
        return f"{self.name} works as a {self.position}."

    # Static Utility Method (no self, acts as a general helper check)
    @staticmethod
    def is_valid_position(position):
        valid_roles = ["manager", "cashier", "cook", "janitor"]
        return position.lower() in valid_roles

# Calling static method directly from class (no instantiation needed!)
print(Employee.is_valid_position("cook")) # Outputs: True
print(Employee.is_valid_position("pilot")) # Outputs: False
```

## Class Methods ( @classmethod )

A **class method** is a method that belongs to the class itself rather than individual object instances [311].

\* **Characteristics:** They accept `cls` as their first parameter (representing the class itself) instead of `self` [311]. This allows them to read or modify class-level variables [312, 314].

\* **Decorator:** Defined using the `@classmethod` decorator [312].

```
class Student:
    num_students = 0
    total_gpa = 0.0

    def __init__(self, name, gpa):
        self.name = name
        self.gpa = gpa
        Student.num_students += 1
        Student.total_gpa += gpa

    # Class method to retrieve class-level metrics
    @classmethod
    def get_average_gpa(cls):
        if cls.num_students == 0:
            return 0.0
        return cls.total_gpa / cls.num_students

# Instantiate students
s1 = Student("Spongebob", 3.2)
s2 = Student("Sandy", 4.0)

# Invoke class method directly from class
print(f"Average Class GPA: {Student.get_average_gpa():.2f}") # Outputs: 3.60
```

# Module 31: Magic/Dunder Methods & Property Decorators

## Magic Methods (Dunder Methods)

**Magic methods** (short for **double underscore** / **dunder** methods) are built-in methods automatically invoked by Python when performing specific operations on objects [316]. You can implement these methods inside your classes to customize how your objects behave with standard operations [316].

Here are the most common dunder methods:

| Method signature                      | Triggered By                                                      | Custom Behavior Purpose                                                    |
|---------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>__init__(self, ...)</code>      | Instantiating an object [317]                                     | Initializes object attributes [318].                                       |
| <code>__str__(self)</code>            | <code>print(object)</code> or <code>str(object)</code> [318, 319] | Returns a clean string representation instead of the memory address [318]. |
| <code>__eq__(self, other)</code>      | <code>object_1 == object_2</code> [319]                           | Defines equality logic [319].                                              |
| <code>__lt__(self, other)</code>      | <code>object_1 &lt; object_2</code> [320]                         | Compares if less than [320].                                               |
| <code>__gt__(self, other)</code>      | <code>object_1 &gt; object_2</code> [321]                         | Compares if greater than [321].                                            |
| <code>__add__(self, other)</code>     | <code>object_1 + object_2</code> [321]                            | Dictates addition outcomes [321].                                          |
| <code>__contains__(self, item)</code> | <code>item in object</code> [322]                                 | Enables membership search lookups [322].                                   |
| <code>__getitem__(self, key)</code>   | <code>object[key]</code> [323]                                    | Enables bracket index/key access on objects [323].                         |

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    # Clean print formatting
    def __str__(self):
        return f'{self.title} by {self.author}'

    # Equality check based on title and author
    def __eq__(self, other):
        return self.title == other.title and self.author == other.author

    # Compare pages
    def __lt__(self, other):
        return self.pages < other.pages

    # Add page counts together
    def __add__(self, other):
        return self.pages + other.pages

    # Check if string exists inside title or author
    def __contains__(self, word):
        return word.lower() in self.title.lower() or word.lower() in self.author.lower()

    # Retrieve details via bracket lookup dictionary styles
    def __getitem__(self, key):
        if key == "title":
            return self.title
        elif key == "author":
            return self.author
        elif key == "pages":
            return self.pages
        return f"Key '{key}' not found."

book_1 = Book("The Hobbit", "J.R.R. Tolkien", 310)
book_2 = Book("Harry Potter", "J.K. Rowling", 223)

print(book_1)           # Outputs: 'The Hobbit' by J.R.R. Tolkien
print(book_1 == book_2) # Outputs: False
print(book_1 > book_2)  # Outputs: True
print(book_1 + book_2)  # Outputs: 533 (Combined pages)
print("hobbit" in book_1) # Outputs: True
print(book_2["author"]) # Outputs: J.K. Rowling
```

`__str__` vs. `__repr__`

The table above only lists `__str__` — but you'll see `__repr__` constantly in

real code and in error tracebacks, so it's worth knowing the distinction:

- `__str__`: the **readable** version, meant for end users — used by `print()` and `str()`.
- `__repr__`: the **unambiguous / developer-facing** version, meant to be precise (ideally something you could paste back into Python to recreate the object) — used by the interactive console, inside collections, and as the *fallback* whenever `__str__` isn't defined.

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f'{self.title} by {self.author}'          # human-friendly

    def __repr__(self):
        return f"Book(title={self.title!r}, author={self.author!r})" # precise

book = Book("Dune", "Frank Herbert")
print(book)      # uses __str__: 'Dune' by Frank Herbert
print([book])    # uses __repr__ inside the list: [Book(title='Dune', author='Frank Herbert')]
```

**Rule of thumb:** always define `__repr__` on your classes (cheap and makes debugging far easier); add `__str__` only if you want a nicer, separate human-facing display.

## The Property Decorator ( `@property` )

The `@property` decorator allows you to define a method as a property, allowing you to access it as if it were a simple attribute **without parentheses** [324, 325].

\* **Getter Method:** Reads the value of a protected attribute [325, 327].

\* **Setter Method:** Validates and writes a value to a protected attribute [325, 327].

\* **Deleter Method:** Safely manages resource deletions [325, 329].

\* **Encapsulation Convention:** Use a single leading underscore (e.g., `self._width`) to indicate an attribute is protected/private and should not be accessed directly [326].

```
class Rectangle:
    def __init__(self, width, height):
        self._width = width    # Encapsulated private variables
        self._height = height # Encapsulated private variables

    # 1. Getter for width
    @property
    def width(self):
        return f"{self._width:.1f} cm"

    # 2. Setter for width with validation checks
    @width.setter
    def width(self, new_width):
        if new_width > 0:
            self._width = new_width
        else:
            print("Validation Error: Width must be greater than zero.")

    # 3. Deleter for width
    @width.deleter
    def width(self):
        print("Deleting width attribute...")
        del self._width

rect = Rectangle(3, 4)

# Access getter like a standard attribute (no parentheses)
print(rect.width) # Outputs: 3.0 cm

# Invoke setter with simple assignment
rect.width = 5 # Successfully changes width to 5
rect.width = -2 # Outputs error; validation blocks assignment

# Safe deletion via deleter method
del rect.width
```

---

## Module 32: Decorators & Exception Handling

---

## Decorators

A **decorator** is a function that extends or modifies the behavior of another function without directly changing its source code [330].

\* **Mechanism**: You pass the original function as an argument into the decorator [330]. The decorator defines an internal **wrapper function** that adds logic before/after executing the original function [331, 332].

\* **Unpacking args inside wrappers**: To ensure a decorator works with any positional ( `*args` ) or keyword ( `**kwargs` ) arguments, pass them into the wrapper signature [335].

```
# Decorator Function
def add_sprinkles(original_function):
    # Wrapper function to intercept and add logic
    def wrapper(*args, **kwargs):
        print("You add sweet sprinkles! 🍦") # Added logic
        return original_function(*args, **kwargs) # Execute original function
    return wrapper # Return the wrapper function

# Base function decorated using '@'
@add_sprinkles
def get_ice_cream(flavor):
    print(f"Here is your {flavor} ice cream! 🍦")

get_ice_cream("vanilla")
```

## Exception Handling

An **exception** is a runtime event that interrupts the normal flow of your program [336]. Unhandled exceptions cause your program to crash [336, 337].

To manage risks safely, Python uses `try`, `except`, and `finally` blocks [337, 339]:

\* **try** : Wrap risky code that might raise exceptions (such as division or file access) [337].

\* **except** : Captures and handles specific exceptions gracefully [337, 338].

\* **ZeroDivisionError**: Triggered by dividing by zero [336].

\* **ValueError**: Triggered by converting an incompatible data type (e.g., `int("pizza")` ) [336].

\* **finally** : A cleanup block that **always executes** regardless of whether exceptions occurred or not [339, 340].

```
# Demonstrating exception handling
try:
    numerator = int(input("Enter a numerator: "))
    denominator = int(input("Enter a denominator: "))
    result = numerator / denominator
    print(f"Result: {result}")
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
except ValueError:
    print("Error: Please input numbers only.")
except Exception as e:
    # Catch-all fallback as a last resort
    print(f"An unexpected error occurred: {e}")
finally:
    print("Executing system cleanup block...") # Always runs
```

## Raising Your Own Exceptions

You aren't limited to catching Python's built-in exceptions — you can trigger one yourself with `raise`, and even define your **own exception types** by subclassing `Exception` (see Module 28 for class basics). This is the direct equivalent of C++'s `throw` with a custom exception class.

```
# 1. Raising a built-in exception manually
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative.")
    return age

try:
    set_age(-5)
except ValueError as e:
    print(f"Caught it: {e}")

# 2. Defining and raising a CUSTOM exception type
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the available balance."""
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"Cannot withdraw ${amount}; balance is only ${balance}.")
    return balance - amount

try:
    withdraw(100, 150)
except InsufficientFundsError as e:
    print(f"Transaction blocked: {e}")
```

Custom exceptions make large programs far more readable — `except InsufficientFundsError:` communicates intent much better than a generic `except Exception:` catch-all.

## Module 33: File Detection & File I/O (Plaintext, JSON, CSV)

### File Detection using the `os` Module

The built-in `os` module allows Python to interact with your operating system's directory and file systems [341].

- **Path Types:**
- **Relative Path:** Looks for files relative to the current working directory [341, 343]. (e.g., `test.txt`) [341]
- **Absolute Path:** Full path starting from root directories [341, 344]. (e.g., `C:/Users/Desktop/test.txt` - use double backslashes `\\` or forward slashes `/` to prevent escaping conflicts) [343, 344].

```
import os

path = "test.txt"

# 1. Check if path exists (folders or files)
if os.path.exists(path):
    print("Path exists!")

# 2. Check if path is a file
if os.path.isfile(path):
    print("This is a file.")

# 3. Check if path is a folder directory
elif os.path.isdir(path):
    print("This is a directory.")
else:
    print("Path does not exist.")
```

### Context Managers ( `with open()` )

When working with file reading or writing, **always use the `with` statement** [346]. This establishes a context manager that automatically closes files once the code block finishes, preventing resource leaks and locking errors [346].

### Writing and Appending Files

Modes:

- \* `'w'`: **Write/Overwrite** mode. Creates the file if it doesn't exist; overrides all contents if it does [346, 347].
- \* `'x'`: **Exclusive write** mode. Safely creates the file, but raises a `FileExistsError` if it already exists [347, 348].
- \* `'a'`: **Append** mode. Adds new data to the end of the existing file without overwriting [347, 349].

```
import json
import csv

# 1. Writing Plaintext (.txt)
employees = ["Spongebob", "Patrick", "Sandy"]
with open("employees.txt", "w") as file:
    for emp in employees:
        file.write(emp + "\n") # Write list items with a newline separation

# 2. Writing JSON (.json)
# Uses the 'json' module to convert Python dictionaries to JSON text strings
user_profile = {"username": "BroCode", "score": 100, "active": True}
with open("profile.json", "w") as file:
    json.dump(user_profile, file, indent=4) # 'indent' formats output cleanly

# 3. Writing CSV (.csv)
# Uses the 'csv' module to write rows of comma-separated variables
# Always set newline="" to prevent blank row gaps between records on Windows
data = [
    ["Name", "Role", "GPA"],
    ["Spongebob", "Cook", 3.2],
    ["Sandy", "Scientist", 4.0]
]
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data) # Writes the entire 2D collection
```

## Reading Files

Always wrap reading calls inside a `try-except` block to capture `FileNotFoundError` or `PermissionError` [356, 357].

```
import json
import csv

# 1. Reading Plaintext (.txt)
try:
    with open("employees.txt", "r") as file:
        content = file.read()
        print(content)
except FileNotFoundError:
    print("Error: Plaintext file was not found.")

# 2. Reading JSON (.json)
try:
    with open("profile.json", "r") as file:
        data = json.load(file) # Converts JSON text back to a Python dictionary
        print(data.get("username"))
except FileNotFoundError:
    print("Error: JSON file was not found.")

# 3. Reading CSV (.csv)
# Reads lines sequentially to build list outputs
try:
    with open("students.csv", "r") as file:
        reader = csv.reader(file)
        for row in reader:
            print(f"Student: {row[0]}, Role: {row[1]}, GPA: {row[2]}")
except FileNotFoundError:
    print("Error: CSV file was not found.")
```

## Module 34: Datetime & Alarm Clock Project

### Dates & Times with Python

Python's built-in `datetime` module allows you to query your computer's system clock [359].

```
import datetime

# 1. Creating manual date objects (Year, Month, Day)
custom_date = datetime.date(2025, 1, 2)
print(custom_date) # Outputs: 2025-01-02

# 2. Getting today's date
today = datetime.date.today()
print(today) # Outputs current system date (e.g., 2026-08-06)

# 3. Getting current system date & time
now = datetime.datetime.now()
print(now)

# 4. Customizing Date/Time Formatting using strftime()
# Format specifiers: %m (month), %d (day), %Y (four-digit year)
# %H (24-hour), %M (minute), %S (seconds), %p (AM/PM)
formatted_now = now.strftime("%m-%d-%Y %I:%M:%S %p")
print(formatted_now) # Outputs: 08-06-2026 04:00:50 AM
```

---

## Project 26: Alarm Clock Application with Pygame Sound

A functional terminal alarm clock. It accepts user-defined military times, continuously monitors system clock states in a secondary running thread, outputs countdowns, and triggers Pygame music files upon hitting matching times [363, 364, 365, 368].

```
# alarm_clock.py
import time
import datetime
# Suppress the Pygame standard console launch message:
import os
os.environ['PYGAME_HIDE_SUPPORT_PROMPT'] = "hide"
import pygame

def trigger_alarm(sound_file_path):
    print("\nALARM TRIGGERED! WAKE UP! 🚨")

    # Initialize the Pygame audio mixer
    pygame.mixer.init()
    # Load and play the MP3 audio file
    pygame.mixer.music.load(sound_file_path)
    pygame.mixer.music.play()

    # Continuous loop to keep the program alive as long as the sound file is playing
    while pygame.mixer.music.get_busy():
        time.sleep(1)

def set_alarm(alarm_target_time, sound_file):
    print(f"Alarm successfully set for: {alarm_target_time}")
    is_running = True

    while is_running:
        # Sleep for exactly 1 second to update the clock tick
        time.sleep(1)

        # Query the system clock time formatted as HH:MM:SS
        current_time = datetime.datetime.now().strftime("%H:%M:%S")
        print(f"Current Time: {current_time}", end="\r")

        # Check for alarm trigger match
        if current_time == alarm_target_time:
            is_running = False
            trigger_alarm(sound_file)

def main():
    print("--- PYTHON ALARM CLOCK ---")
    # Prompt user for target time in 24-hour military formatting
    alarm_time = input("Enter alarm time in HH:MM:SS format (e.g. 13:30:00): ")

    # Ensure you place a valid MP3 file in your project folder
    sound_path = "alarm_sound.mp3"

    # Create a placeholder dummy file if it doesn't exist to prevent Pygame load errors
    if not os.path.exists(sound_path):
        with open(sound_path, "w") as f:
            f.write("") # Create empty file to bypass early OS checks

    set_alarm(alarm_time, sound_path)

if __name__ == '__main__':
    main()
```

## Module 35: Multi-threading & API Pokémon Integration

### Multi-Threading in Python

Normally, your Python programs execute code line-by-line sequentially on a single CPU thread (the main thread) [372]. **Multi-threading** allows your program to perform multiple tasks concurrently (multitasking) [370].

- **Use Case:** Best for **I/O Bound** operations (tasks that wait on inputs/outputs, such as reading massive files, querying databases, or loading API responses) [370, 376].
- **Unpacking Parameters:** When spinning up threads, target arguments must be packed as a **tuple** passed to the `args` parameter [375].
- *Important syntax detail:* If passing a singular argument, **you must add a trailing comma** inside the parentheses (e.g., `args=(arg_1,)` ), otherwise Python won't recognize it as a tuple [375].

```
import threading
import time

def perform_chore(name, duration):
    print(f"Started chore: {name}")
    time.sleep(duration) # Simulates waiting for I/O operations
    print(f"Finished chore: {name}")

# Create concurrent threads
thread_1 = threading.Thread(target=perform_chore, args=("Walk Dog", 4))
thread_2 = threading.Thread(target=perform_chore, args=("Wash Dishes", 2))

# 1. Start threads (they run concurrently)
thread_1.start()
thread_2.start()

# 2. Join threads (forces the main script to wait until threads finish before moving on)
thread_1.join()
thread_2.join()

print("All chores completed successfully!")
```

## API Integrations using `requests`

An **API (Application Programming Interface)** allows programs to fetch and exchange data over the internet [376].

\* To query endpoints in Python, use the external `requests` library [376, 377].

\* Run `pip install requests` in your terminal to install the package [377].

\* **HTTP Response Codes:**

\* `200` : Success (Data retrieved) [379].

\* `404` : Resource Not Found [379].

```
import requests

def get_pokemon_stats(pokemon_name):
    base_url = "https://pokeapi.co/api/v2/pokemon/"
    # Complete the request URL
    url = f"{base_url}{pokemon_name.lower()}"

    # Send GET request to endpoint
    response = requests.get(url)

    if response.status_code == 200:
        # Convert JSON text payload into a Python dictionary
        data = response.json()

        # Extract specific data keys
        print(f"Name:    {data['name'].capitalize()}")
        print(f"ID:      {data['id']}")
        print(f"Height: {data['height']}")
        print(f"Weight: {data['weight']}")
    else:
        print(f"Error: Failed to retrieve data. HTTP Code: {response.status_code}")

get_pokemon_stats("pikachu")
```

## Virtual Environments ( `venv` )

Once you install third-party packages ( `requests` , `PyQt5` , etc.), different projects will inevitably need **different, sometimes conflicting, versions** of the same library. A **virtual environment** is an isolated, self-contained Python install-and-package-set for a single project — the rough equivalent of keeping separate `node_modules` folders per project in JS, or separate build configurations per project in C++.

```
# 1. Create a virtual environment (creates a 'venv' folder in your project)
python -m venv venv

# 2. Activate it
# Windows:
venv\Scripts\activate
# macOS/Linux:
source venv/bin/activate

# 3. Install packages – they install into THIS environment only
pip install requests PyQt5

# 4. Deactivate when done
deactivate
```

**Why it matters:** without a virtual environment, every `pip install` goes into your one global, system-wide Python — and two projects requiring different versions of the same library will silently break each other. PyCharm can create and manage a `venv` for you automatically per-project (it's typically offered during "New Project" setup back in Module 1).

---

## Module 36: PyQt5 GUI Development Fundamentals

---

### PyQt5 Introduction

**PyQt5** is a comprehensive binding library for the Qt framework, used to develop cross-platform **Graphical User Interfaces (GUIs)** [381, 382].

- **Installation:** Run `pip install PyQt5` in your terminal [382].
- **Core Widgets & Imports:**
  - `QApplication`: Manages the GUI application's control flow, system settings, and initialization loops [383]. Every PyQt5 app **must** have exactly one `QApplication` instance [384].
  - `QMainWindow`: Provides a standard main window framework, accommodating menu bars, status bars, and central widgets [383, 399].

### PyQt5 Window Boilerplate Architecture

Below is the standard, production-ready class architecture used to spin up and maintain window displays:

```
# pyqt5_window.py
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow
from PyQt5.QtGui import QIcon

# Create custom Window class inheriting from QMainWindow
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        # Call UI styling initialization method
        self.init_ui()

    def init_ui(self):
        # 1. Set Title Bar
        self.setWindowTitle("My First Python GUI")

        # 2. Configure Geometry: set_geometry(X_pos, Y_pos, width, height)
        # X and Y represent where the window launches on your screen
        self.setGeometry(700, 300, 500, 500)

        # 3. Add Window Icon
        # Make sure you have a valid image in your directory, e.g., 'logo.png'
        self.setWindowIcon(QIcon("logo.png"))

def main():
    # Instantiate the single core application loop
    app = QApplication(sys.argv)

    # Construct the window
    window = MainWindow()

    # Windows are hidden by default; explicitly invoke show()
    window.show()

    # sys.exit ensures a clean, error-free system shutdown when the window is closed
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

---

## Module 37: PyQt5 Widgets (Buttons, Checkboxes, Radios, Textboxes)

---

### Core PyQt5 Widgets Reference

To design interactive, feature-rich interfaces, Python developers leverage various GUI widgets:

1. **Labels ( `QLabel` ):** Displays read-only plaintext or images [388, 394].
  - \* *Styles:* Style widgets using CSS-like properties with `.setStyleSheet("CSS_rules")` [390, 422].
  - \* *Alignment:* Align text horizontally and vertically using the `Qt` alignment class (e.g.,

```
self.label.setAlignment(Qt.AlignCenter) ) [391].
```

```
from PyQt5.QtWidgets import QLabel
from PyQt5.QtGui import QFont, QPixmap
from PyQt5.QtCore import Qt

# Plaintext label with alignment and font
label = QLabel("Hello User", self)
label.setFont(QFont("Arial", 40))
label.setGeometry(0, 0, 500, 100)
label.setAlignment(Qt.AlignCenter) # Align both horizontally and vertically

# Applying CSS styling
label.setStyleSheet("color: #2e2e2e; background-color: lightblue; font-weight: bold;")

# Image labels (Using QPixmap)
image_label = QLabel(self)
pixmap = QPixmap("image.png")
image_label.setPixmap(pixmap)
image_label.setScaledContents(True) # Scale image automatically to fit label dimensions
```

## 2. **Layout Managers ( QBoxLayout , QHBoxLayout , QGridLayout ):** Controls widget positioning automatically [398, 401].

\* Since you cannot apply layout managers directly to `QMainWindow`, you must create a generic container `QWidget` (central widget), set its layout, and assign it to the window [399].

```
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QPushButton

# Main Central Widget Container Setup
central_widget = QWidget()
self.setCentralWidget(central_widget)

# Setup Vertical Layout
layout = QVBoxLayout()
layout.addWidget(QPushButton("Button 1"))
layout.addWidget(QPushButton("Button 2"))

# Assign layout to central widget
central_widget.setLayout(layout)
```

## 3. **Buttons ( QPushButton ):** Trigger actions when clicked [402, 404]. Connect clicks (signals) to functions (slots) using `.clicked.connect()` [404].

```
button = QPushButton("Click Me", self)
# Connect click event (signal) to local slot method
button.clicked.connect(self.on_button_clicked)
button.setDisabled(True) # Disables button interactions
```

## 4. **Checkboxes ( QCheckBox ):** Binary choice selectors [407]. Track changes using `.stateChanged.connect()` [409, 410].

\* State evaluations: unchecked is `0` ( `Qt.Unchecked` ); checked is `2` ( `Qt.Checked` ) [410, 411].

```
from PyQt5.QtWidgets import QCheckBox

checkbox = QCheckBox("Agree to Terms", self)
checkbox.stateChanged.connect(self.on_state_changed)

def on_state_changed(self, state):
    if state == Qt.Checked:
        print("User checked the box.")
```

## 5. **Radio Buttons ( QRadioButton & QButtonGroup ):** Mutually exclusive selection groups [414].

\* **Important Rule:** By default, all radio buttons in a window belong to a single group [414]. To allow multiple independent groups (e.g., separating "Payment Method" from "Delivery Method"), wrap them in separate `QButtonGroup` instances [414, 415, 416].

```
from PyQt5.QtWidgets import QRadioButton, QButtonGroup

# Group 1
group_1 = QButtonGroup(self)
radio_1 = QRadioButton("Visa", self)
radio_2 = QRadioButton("Mastercard", self)
group_1.addButton(radio_1)
group_1.addButton(radio_2)
```

## 6. **LineEdit Textboxes ( QLineEdit ):** Single-line text input fields [418]. Extract content using `.text()`; set hints using `.setPlaceholderText()` [421, 422].

```
text_input = QLineEdit(self)
text_input.setPlaceholderText("Enter your name...")

# Get user input string
username = text_input.text()
```

## Module 38: PyQt5 Digital Clock Widget Project

### Project 27: PyQt5 Digital Clock Widget

A functional desktop clock. It inherits from `QWidget`, implements vertical layouts, formats date-time objects, triggers updates every 1000 milliseconds using `QTimer` [430], and dynamically loads custom TTF digital fonts into the application database [438, 439].

```
# digital_clock.py
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QLabel, QVBoxLayout
from PyQt5.QtCore import QTimer, QTime, Qt
from PyQt5.QtGui import QFont, QFontDatabase

class DigitalClock(QWidget):
    def __init__(self):
        super().__init__()
        self.init_ui()

    def init_ui(self):
        self.setWindowTitle("Python Digital Clock")
        self.setGeometry(700, 300, 400, 150)
        self.setStyleSheet("background-color: black;")

        # Step 1: Create layout and text label
        self.time_label = QLabel("00:00:00 AM", self)
        self.time_label.setAlignment(Qt.AlignCenter)

        # Apply neon-green coloring and sizing styles
        self.time_label.setStyleSheet("color: #00FF00;")

        # Step 2: Configure layout hierarchy
        layout = QVBoxLayout()
        layout.addWidget(self.time_label)
        self.setLayout(layout)

        # Step 3: Implement QTimer to run updates every second
        self.timer = QTimer(self)
        self.timer.timeout.connect(self.update_time)
        self.timer.start(1000) # Trigger 'timeout' signal every 1000 milliseconds

        # Step 4: Add custom TTF digital font
        # Place digital font file in the project folder, e.g. 'DS-DIGI.TTF'
        font_id = QFontDatabase.addApplicationFont("DS-DIGI.TTF")
        if font_id != -1:
            font_families = QFontDatabase.applicationFontFamilies(font_id)
            self.time_label.setFont(QFont(font_families[0], 120))
        else:
            self.time_label.setFont(QFont("Arial", 80, QFont.Bold))

        self.update_time() # Run once immediately on start

    def update_time(self):
        # Retrieve the current system time
        current_time = QTime.currentTime().toString("hh:mm:ss AP")
        self.time_label.setText(current_time)

def main():
    app = QApplication(sys.argv)
    clock = DigitalClock()
    clock.show()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

## Module 39: PyQt5 Precision Stopwatch App Project

### Project 28: PyQt5 Precision Stopwatch App

A precision stopwatch [441]. It handles times down to the millisecond by triggering `QTimer` timeouts every 10ms [449], formats H:M:S:MS with zero-padding format specifiers [450], and implements Start, Stop, and Reset controls [444].

```
# precision_stopwatch.py
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QLabel, QPushButton, QVBoxLayout, QHBoxLayout
from PyQt5.QtCore import QTimer, QTime, Qt
from PyQt5.QtGui import QFont

class Stopwatch(QWidget):
    def __init__(self):
        super().__init__()
        self.init_ui()

    def init_ui(self):
        self.setWindowTitle("Stopwatch")
        self.setGeometry(700, 300, 450, 250)
        self.setStyleSheet("background-color: #1e1e1e;")

        # Step 1: Initialize baseline variables
        self.time = QTime(0, 0, 0, 0) # Hours, Minutes, Seconds, Milliseconds

        # Step 2: Construct GUI layout widgets
        self.time_label = QLabel("00:00:00.00", self)
        self.time_label.setAlignment(Qt.AlignCenter)
        self.time_label.setFont(QFont("Calibri", 100, QFont.Bold))
        self.time_label.setStyleSheet("color: white; background-color: #002b5c; border-radius: 20px; padding: 20px;")

        self.start_btn = QPushButton("Start", self)
        self.stop_btn = QPushButton("Stop", self)
        self.reset_btn = QPushButton("Reset", self)

        # Apply HSL CSS formatting across buttons
        button_style = """
        QPushButton {
            font-size: 30px;
            font-weight: bold;
            font-family: Calibri;
            color: white;
            background-color: hsl(210, 60%, 40%);
            padding: 15px;
            border-radius: 10px;
        }
        QPushButton:hover {
            background-color: hsl(210, 60%, 55%);
        }
        """
        self.start_btn.setStyleSheet(button_style)
        self.stop_btn.setStyleSheet(button_style)
        self.reset_btn.setStyleSheet(button_style)

        # Step 3: Align layouts
        # Buttons are aligned horizontally
        btn_layout = QHBoxLayout()
        btn_layout.addWidget(self.start_btn)
        btn_layout.addWidget(self.stop_btn)
        btn_layout.addWidget(self.reset_btn)

        # Everything (label + button row) is aligned vertically
        main_layout = QVBoxLayout()
        main_layout.addWidget(self.time_label)
        main_layout.addLayout(btn_layout)
        self.setLayout(main_layout)

        # Step 4: Configure Timer and connect signals to slot actions
        self.timer = QTimer(self)
        self.timer.timeout.connect(self.update_display)

        self.start_btn.clicked.connect(self.start_stopwatch)
        self.stop_btn.clicked.connect(self.stop_stopwatch)
        self.reset_btn.clicked.connect(self.reset_stopwatch)

    def start_stopwatch(self):
        self.timer.start(10) # Trigger timeout every 10 milliseconds

    def stop_stopwatch(self):
        self.timer.stop()

    def reset_stopwatch(self):
        self.timer.stop()
        self.time = QTime(0, 0, 0, 0)
        self.time_label.setText("00:00:00.00")

    def update_display(self):
        # Increment time object by 10 milliseconds
```

```

self.time = self.time.addMsecs(10)

# Extract individual components
hours = self.time.hour()
minutes = self.time.minute()
seconds = self.time.second()
# Divide by 10 to truncate 3 digit milliseconds to 2 digits
milliseconds = self.time.msec() // 10

# Update label text using formatted string specifiers
self.time_label.setText(f"{hours:02}:{minutes:02}:{seconds:02}.{milliseconds:02}")

def main():
    app = QApplication(sys.argv)
    stopwatch = Stopwatch()
    stopwatch.show()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()

```

## Module 40: PyQt5 Real-Time Weather Application Project

### Project 29: PyQt5 Real-Time Weather Application

The capstone project: a real-time PyQt5 weather desktop client [452]. It fetches weather metrics from the OpenWeatherMap API [452], handles networking and API error states inside a custom `match-case` statement [469], and uses a static method to map OpenWeatherMap ID ranges to colorful custom emojis [479, 481].

```

# weather_app.py
import sys
import requests
from PyQt5.QtWidgets import QApplication, QWidget, QLabel, QLineEdit, QPushButton, QVBoxLayout
from PyQt5.QtCore import Qt, QTimer
from PyQt5.QtGui import QFont

class WeatherApp(QWidget):
    def __init__(self):
        super().__init__()
        self.init_ui()

    def init_ui(self):
        self.setWindowTitle("Weather App Client")
        self.setGeometry(700, 300, 450, 600)
        self.setStyleSheet("background-color: #2b2b2b;")

        # Step 1: Create layout widgets and configure object names for target styling
        self.title_label = QLabel("Enter City Name:", self)
        self.title_label.setObjectName("city_label")

        self.city_input = QLineEdit(self)
        self.city_input.setPlaceholderText("e.g. Miami, Paris, Tokyo...")
        self.city_input.setObjectName("city_input")

        self.get_weather_btn = QPushButton("Get Weather", self)
        self.get_weather_btn.setObjectName("get_weather_btn")

        self.temp_label = QLabel(self)
        self.temp_label.setObjectName("temp_label")
        self.temp_label.setAlignment(Qt.AlignCenter)

        self.emoji_label = QLabel(self)
        self.emoji_label.setObjectName("emoji_label")
        self.emoji_label.setAlignment(Qt.AlignCenter)

        self.desc_label = QLabel(self)
        self.desc_label.setObjectName("desc_label")
        self.desc_label.setAlignment(Qt.AlignCenter)

        # Step 2: Configure Layout Hierarchy
        layout = QVBoxLayout()
        layout.addWidget(self.title_label, alignment=Qt.AlignCenter)
        layout.addWidget(self.city_input)
        layout.addWidget(self.get_weather_btn)
        layout.addWidget(self.temp_label, alignment=Qt.AlignCenter)
        layout.addWidget(self.emoji_label, alignment=Qt.AlignCenter)
        layout.addWidget(self.desc_label, alignment=Qt.AlignCenter)
        self.setLayout(layout)

        # Step 3: Advanced Stylesheets using selectors

```

```

self.setStyleSheet("""
    QWidget {
        background-color: #222222;
    }
    QLabel {
        font-family: Calibri;
        color: #ffffff;
    }
    QLabel#city_label {
        font-size: 35px;
        font-style: italic;
    }
    QLineEdit#city_input {
        font-size: 30px;
        font-family: Calibri;
        padding: 10px;
        color: black;
        background-color: white;
        border-radius: 5px;
    }
    QPushButton#get_weather_btn {
        font-size: 28px;
        font-weight: bold;
        font-family: Calibri;
        color: white;
        background-color: #0078D7;
        padding: 10px;
        border-radius: 5px;
    }
    QPushButton#get_weather_btn: hover {
        background-color: #005A9E;
    }
    QLabel#temp_label {
        font-size: 75px;
        font-weight: bold;
    }
    QLabel#emoji_label {
        font-size: 100px;
        font-family: "Segoe UI Emoji";
    }
    QLabel#desc_label {
        font-size: 35px;
        font-style: italic;
    }
""")

# Connect button signals
self.get_weather_btn.clicked.connect(self.get_weather)

def display_error(self, message):
    # Format label sizing to display errors clearly
    self.temp_label.setFont(QFont("Calibri", 20, QFont.Bold))
    self.temp_label.setText(message)
    self.emoji_label.clear()
    self.desc_label.clear()

def get_weather(self):
    city = self.city_input.text().strip()
    if not city:
        self.display_error("Bad Request: Please enter a city name.")
        return

    # Secure active target API credentials
    api_key = "INSERT_YOUR_API_KEY_HERE"
    url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}"

    try:
        response = requests.get(url, timeout=5)
        # Raise exception immediately on HTTP error status codes
        response.raise_for_status()

        # Convert payload to dictionary
        data = response.json()
        self.display_weather(data)

    except requests.exceptions.HTTPError:
        # Capture HTTP Status Codes to display custom descriptions using match-case
        status_code = response.status_code
        match status_code:
            case 400:
                self.display_error("Bad Request: Please check input formatting.")
            case 401:
                self.display_error("Unauthorized: Invalid API Key configuration.")
            case 403:
                self.display_error("Forbidden: Access denied.")

```

```

        case 404:
            self.display_error("Error: City not found in directory.")
        case 500:
            self.display_error("Server Error: Please try again later.")
        case _:
            self.display_error(f"HTTP Error Occurred: Status Code {status_code}")

    except requests.exceptions.ConnectionError:
        self.display_error("Connection Error: Check your internet connection.")
    except requests.exceptions.Timeout:
        self.display_error("Timeout Error: Server took too long to respond.")
    except Exception as e:
        self.display_error(f"Error: {e}")

def display_weather(self, data):
    # Reset default fonts
    self.temp_label.setFont(QFont("Calibri", 75, QFont.Bold))

    # 1. Extract temperature in Kelvin and convert to Fahrenheit
    temp_k = data['main']['temp']
    temp_f = (temp_k * 9/5) - 459.67 # Kelvin to Fahrenheit
    # For Celsius: temp_c = temp_k - 273.15

    # 2. Extract description and ID
    weather_id = data['weather'][0]['id']
    description = data['weather'][0]['description'].capitalize()

    # 3. Update GUI elements
    self.temp_label.setText(f"{temp_f:.0f}°F")
    self.desc_label.setText(description)

    # Look up correct weather emoji using our static method
    emoji = self.get_weather_emoji(weather_id)
    self.emoji_label.setText(emoji)

@staticmethod
def get_weather_emoji(weather_id):
    # Static helper method mapping OpenWeatherMap ranges to emojis
    if 200 <= weather_id <= 232:
        return "⚡" # Thunderstorm
    elif 300 <= weather_id <= 321:
        return "🌧️" # Drizzle
    elif 500 <= weather_id <= 531:
        return "🌧️" # Rain
    elif 600 <= weather_id <= 622:
        return "❄️" # Snow
    elif 701 <= weather_id <= 741:
        return "🌫️" # Mist/Fog
    elif weather_id == 762:
        return "🌋" # Volcanic Ash
    elif weather_id == 771:
        return "🌩️" # Squall
    elif weather_id == 781:
        return "🌪️" # Tornado
    elif weather_id == 800:
        return "☀️" # Clear Sky
    elif 801 <= weather_id <= 804:
        return "☁️" # Clouds
    return "?"

def main():
    app = QApplication(sys.argv)
    weather_widget = WeatherApp()
    weather_widget.show()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()

```

## Appendix: Common Pitfalls Cheat-Sheet

A fast-scan reference of every trap covered in this document, in one place — useful to re-read once you've finished all 40 modules.

| Pitfall                                     | Wrong Instinct (often from C/C++)                            | Correct Python Behavior                                                                                  |
|---------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Copying a list                              | <code>b = a</code> copies the data                           | <code>b = a</code> copies the <i>reference</i> ; use <code>.copy()</code> / <code>copy.deepcopy()</code> |
| Comparing values                            | <code>a is b</code> checks equality                          | <code>is</code> checks identity; use <code>==</code> for value equality                                  |
| Default mutable args                        | <code>def f(x=[])</code> gives a fresh list each call        | The default list is created ONCE and shared across calls                                                 |
| 2D list initialization                      | <code>[[0]*3]*3</code> makes 3 independent rows              | All 3 "rows" are the same shared list object                                                             |
| Modifying a variable from an inner function | Inner scopes can freely write to outer variables             | Assignment creates a new local unless you use <code>global</code> / <code>nonlocal</code>                |
| Looping with an index                       | <code>for i in range(len(x))</code> then <code>x[i]</code>   | Prefer <code>enumerate(x)</code> ; use <code>zip()</code> for parallel sequences                         |
| Checking for <code>None</code>              | <code>if x == None:</code>                                   | Idiomatic Python is <code>if x is None:</code>                                                           |
| Float division                              | <code>/</code> truncates like C's <code>int/int</code>       | <code>/</code> always returns a <code>float</code> ; use <code>//</code> for integer (floor) division    |
| String immutability                         | <code>s[0] = 'A'</code> to mutate a string in place          | Strings are immutable — build a new string instead                                                       |
| Function overloading                        | Redefine <code>def f(x)</code> then <code>def f(x, y)</code> | The second definition silently replaces the first                                                        |

## What This Document Doesn't Cover (Your Next Steps After Mastering This)

This guide faithfully covers a thorough 40-module beginner-to-intermediate Python course, plus the conceptual gaps a C/C++ background tends to hit. Once you're comfortable with everything above, these are the natural next topics — intentionally out of scope here because they go beyond "beginner course" material:

- **Type hints** ( `def add(x: int, y: int) -> int:` ) — optional static typing annotations, increasingly standard in professional codebases, checked by tools like `mypy` rather than the interpreter itself.
- **Generators & `yield`** — functions that produce a sequence of values lazily, one at a time, instead of building a full list in memory. Foundational for working with large datasets efficiently.
- **`async` / `await` & `asyncio`** — Python's alternative to threading (Module 35) for I/O-bound concurrency, dominant in modern web servers and network clients.
- **Testing** ( `pytest` / `unittest` ) — writing automated tests for your code, the professional-grade equivalent of manually re-running your scripts to check they still work.
- **Packaging & distribution** — turning a project into an installable package ( `pyproject.toml` , publishing to PyPI).
- **Web frameworks** (Flask, FastAPI, Django) — natural next step after the API integration work in Module 35, but building the server side instead of just consuming one.
- **Data science stack** ( `numpy` , `pandas` , `matplotlib` ) — if your interests lean toward data/ML rather than GUI apps (Modules 36–40), this is the more common professional direction.

You now have a genuinely complete, single-document reference spanning setup through GUI application development, with every major beginner-to-intermediate concept, syntax, gotcha, and C/C++-to-Python translation note included in the right chronological place. Read it once end-to-end, then keep it as your lookup reference while you build things.